Mini project report on

SOFTWARE DEPENDENCY VULNERABILITY AND IMPACT TRACKING SYSTEM (SDVIT)

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Name	SRN
Samaresh	PES2UG24CS657
Vaibhav R	PES2UG24CS664

under the guidance of
Dr. Shilpa S
Assistant Professor

PES University
AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

SOFTWARE DEPENDENCY VULNERABILITY AND IMPACT TRACKING SYSTEM (SDVIT)

is a bonafide work carried out by

Name	**SRN**
Samaresh	PES2UG24CS657
Vaibhav R	PES2UG24CS664

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature

Dr. Shilpa S, Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled Software Dependency Vulnerability and Impact Tracking System (SDVIT) has been carried out by us under the guidance of Dr. Shilpa S, Assistant Professor, and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester AUG – DEC 2026. The work presented in this report is based on our project implementation and has not been submitted elsewhere for any other academic award.

Name	**SRN**	**Signature**
Samaresh	PES2UG24CS657	
Vaibhav R	PES2UG24CS664	

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

We take this opportunity to thank the Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

ABSTRACT

Software applications rely on third-party libraries and packages to provide functionality and reduce development effort. However, these dependencies may contain security vulnerabilities that expose applications to risk. Identifying vulnerable dependency versions and understanding which projects are affected are important tasks in software security management.

The Software Dependency Vulnerability and Impact Tracking System (SDVIT) organizes software projects, packages, package versions, dependencies, vulnerabilities, affected versions, alerts, and project risk information in a single relational schema. CVE identifiers identify publicly known vulnerabilities and CVSS scores represent their severity.

The database is implemented in MySQL and contains nine tables connected through primary keys, foreign keys, unique constraints, CHECK constraints and indexes. Two views, stored procedures, stored functions and triggers support vulnerability reporting, severity classification, automatic alert generation, alert management (with transactions) and project risk snapshots. The frontend is built with Next.js and React and the backend with Node.js and Express. A Neo4j graph database is used for transitive dependency analysis and "blast radius" queries.

SDVIT provides a centralized approach to dependency and vulnerability tracking, helping developers identify affected projects, monitor security alerts, and assess project exposure.

TABLE OF CONTENTS

1. INTRODUCTION 8
1.1 Technology used 8
1.2 Why two databases 8
2. PROBLEM DEFINITION 9
2.1 Problem statement 9
2.2 Existing system 9
2.3 Proposed system 9
2.4 Objectives 9
2.5 Scope 10
2.6 Functional requirements 10
3. ER MODEL 11
4. ER TO RELATIONAL MAPPING 14
4.1 Steps of algorithm for the chosen problem 14
4.2 Relational mapping table 14
4.3 Complete diagram of relational mapping 15
5. DDL STATEMENTS 16
5.1 CREATE TABLE statements 16
5.2 ALTER, INDEX and DROP statements 23
6. DML STATEMENTS 24
6.1 Rows loaded per table 24
6.2 Sample data 25
7. QUERIES 28
7.1 Simple query with GROUP BY, aggregate 28
7.2 UPDATE operation 28
7.3 DELETE operation 29
7.4 Correlated query 29
7.5 Nested query 30
8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS 31
8.1 Stored procedures or functions 31
8.2 Triggers 36
8.3 View 40
8.4 Summary of advanced database features 42
9. FRONT END DEVELOPMENT 43
9.1 Overview and architecture 43
9.2 Authentication and roles 43
9.3 Pages 43

9.4 API layer 49
9.5 Validation and security 50
9.6 Running the application 51
CONCLUSION 52
REFERENCES 53
APPENDIX A – DEFINITIONS, ACRONYMS AND ABBREVIATIONS 54

1. INTRODUCTION

Modern software applications are rarely developed entirely from scratch. Developers frequently depend on external libraries, frameworks, and packages to implement functionality efficiently. These third-party components reduce development time and improve productivity. However, dependencies introduce security risk because a vulnerability in a package may affect every project that uses an affected version.

A software dependency is an external component required by a project. A package has many released versions, and a vulnerability may affect only some of them. Storing package names separately from package versions is therefore essential for accurate vulnerability tracking. Keeping this information in manifests, advisories and spreadsheets makes it hard to guarantee consistency or to trace which projects are exposed.

SDVIT builds a relational database for dependency and vulnerability tracking. It stores users, the projects they own, packages and their versions, the dependencies of each project, vulnerabilities (identified by CVE and scored by CVSS), the package versions each vulnerability affects, alerts, and historical project risk snapshots. The design is carried through every stage: ER model, relational mapping, DDL, DML, queries, and advanced features such as views, triggers, stored procedures, functions and transactions.

1.1 Technology used

- Database: MySQL 8.x (accessed from Node.js through the mysql2 driver 3.24)
- Graph database: Neo4j 5.x with Cypher (neo4j-driver 5.28), used for transitive dependency analysis
- Backend / API layer: Node.js 20.19 or later with Express 4.22 (plus cors and dotenv)
- Frontend: Next.js 16 (App Router) with React 19, React Router 6, Axios 1.x for API calls and Recharts 3 for charts
- Design tools: Lucid chart
- Authentication: salted scrypt password hashes (Node.js crypto) and HMAC-SHA256 signed bearer tokens in JWT format with an 8-hour expiry; two roles, admin and developer

1.2 Why two databases

Data	Store	Reason
Users, projects, packages, versions, dependencies, vulnerabilities, affected versions, alerts, risk snapshots	MySQL	Highly relational with many-to-many links; needs foreign keys, constraints, transactions, triggers and joins.
Dependency graph (project → package version → package version …)	Neo4j	Transitive dependencies are variable-depth path traversals, which are natural in a graph and awkward in SQL joins.

2. PROBLEM DEFINITION

2.1 Problem statement

Software projects use multiple third-party packages, and vulnerabilities may exist in particular versions of those packages. Developers need to determine which package versions are affected and which projects depend on them. When dependency and vulnerability information is maintained separately, identifying affected projects and tracking remediation becomes difficult. SDVIT addresses this problem by maintaining the relevant information in a centralized database and supporting alert generation and project risk assessment.

2.2 Existing system

In a basic or manual approach, developers inspect package manifests, vulnerability advisories and project records separately. This requires repeated effort to match package versions with affected-version information and to determine which projects are exposed. It is also difficult to maintain a consistent alert status and historical project risk information across multiple projects.

2.3 Proposed system

SDVIT organizes project, package, version, dependency, vulnerability, affected-version, alert and risk information in a MySQL relational database. Foreign keys and uniqueness constraints maintain valid relationships. Triggers create alerts for high-severity vulnerabilities in two situations: when a project adds a package version already known to be vulnerable, and when a vulnerability is mapped to a package version already used by projects. Stored procedures support reporting, alert status changes and risk snapshot generation. A Neo4j graph supports transitive dependency analysis, and the web frontend provides an interface for all of this.

2.4 Objectives

- Maintain information about users and software projects.
- Store package names and their individual released versions.
- Record project-to-package-version dependency relationships.
- Maintain vulnerability records using CVE identifiers and CVSS scores.
- Map vulnerabilities to the package versions they affect.
- Generate alerts automatically when a project is exposed to a high-severity vulnerability.
- Support alert acknowledgement and resolution as atomic transactions.
- Calculate and retain project risk snapshots.
- Enforce data integrity using primary keys, foreign keys, unique constraints, CHECK constraints and indexes.
- Demonstrate advanced database features: views, triggers, stored procedures, functions, transactions and indexes.

2.5 Scope

The system covers users and projects, packages and versions from the ecosystems npm, PyPI, Maven, Cargo and other, direct project dependencies, vulnerability records (CVE, CVSS, published date), affected-version mappings, alerts with status, and risk snapshots. A Neo4j graph is used for transitive dependency traversal; transitive links are loaded from npm package-lock.json files (version 2 or 3) or added manually. Automatic scanning of real package manifests or live CVE feeds is out of scope; the data is a synthetic/sample dataset.

2.6 Functional requirements

- Create and manage users and projects.
- Maintain a catalogue of packages and their versions with ecosystem information.
- Record which package versions each project depends on.
- Maintain vulnerabilities (CVE id, description, CVSS score, published date) and the package versions they affect.
- Automatically raise an alert when a project becomes exposed to a vulnerability with CVSS ≥ 7.0.
- Acknowledge and resolve alerts; record risk snapshots when alerts are resolved.
- Classify vulnerabilities by severity (Critical / High / Medium / Low) from the CVSS score.
- Provide a web interface with a dashboard, project, package, vulnerability and alert pages.
- Provide a dependency graph view for transitive impact analysis (Neo4j).
- Authenticate users and enforce role-based access (admin / developer) on the server.

3. ER MODEL

The ER model identifies the following entities and relationships. Package version is a weak entity of Package (a version cannot exist without its package) and Risk snapshot is a weak entity of Project. Project and Vulnerability are related many-to-many through Alert, and Project and Package version are related many-to-many through the dependency relationship.

Entity	Type	Key attribute	Purpose
USERS	Strong	id	User details and role information.
PROJECTS	Strong	id	Projects and their owners.
PACKAGES	Strong	id	Package names and ecosystems.
PACKAGE_VERSIONS	Weak (owner: Package)	id	Individual versions belonging to packages.
VULNERABILITIES	Strong	id (cve_id unique)	CVE identifiers, descriptions, CVSS scores and dates.
RISK_SNAPSHOTS	Weak (owner: Project)	id	Historical project risk scores.

Relationship	Entities	Cardinality	Relational table
owns	User – Project	1 : N	FK owner_id
has version	Package – Package version	1 : N	FK package_id
depends on	Project – Package version	M : N	PROJECT_DEPENDENCIES
affects	Vulnerability – Package version	M : N	VULN_AFFECTED_VERSIONS
alerts	Project – Vulnerability	M : N	ALERTS
has	Project – Risk snapshot	1 : N	FK project_id

Vulnerability impact is traced through package versions and project dependencies; the schema does not define a direct foreign key from projects to affected versions.

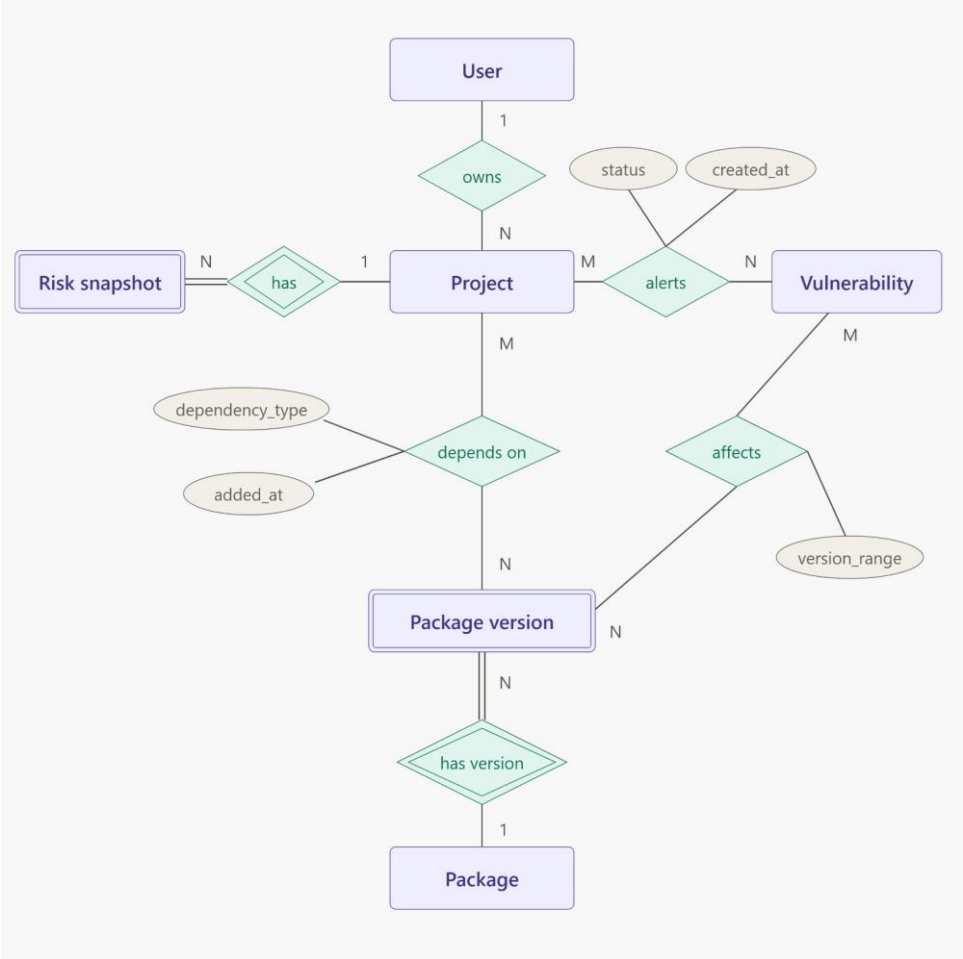


Figure 3.1: ER diagram – entities and relationship cardinalities

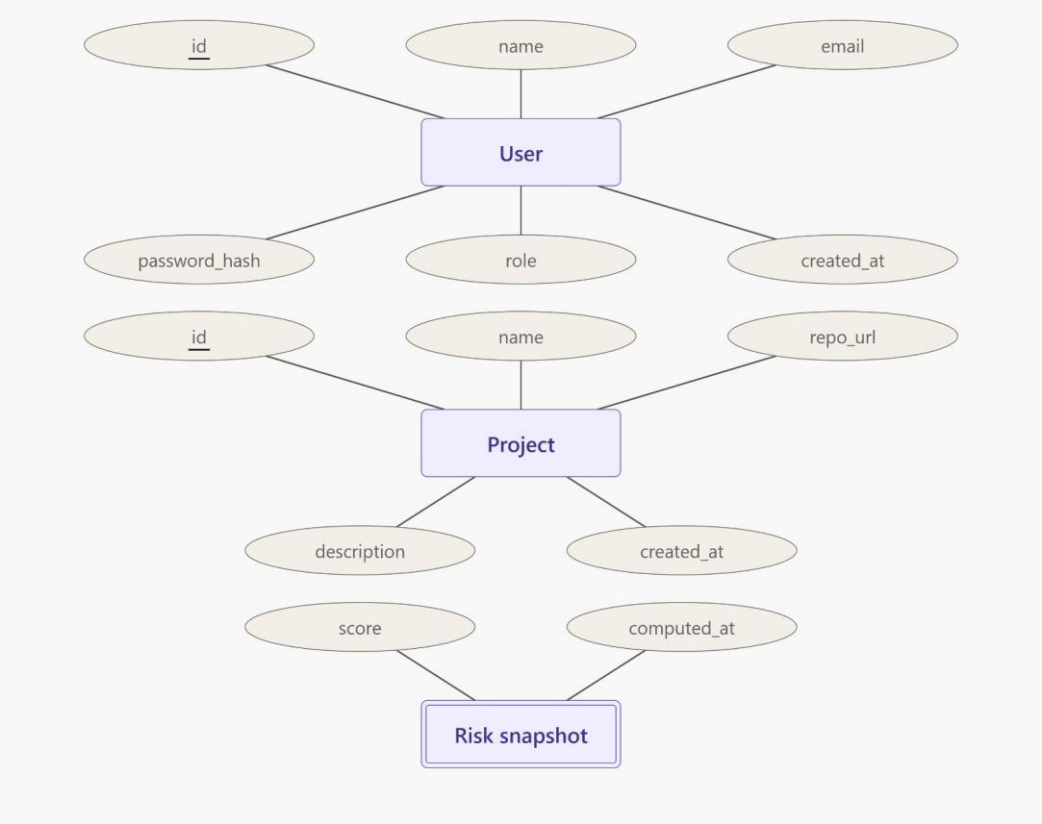


Figure 3.2: ER diagram – attributes of User, Project and Risk snapshot

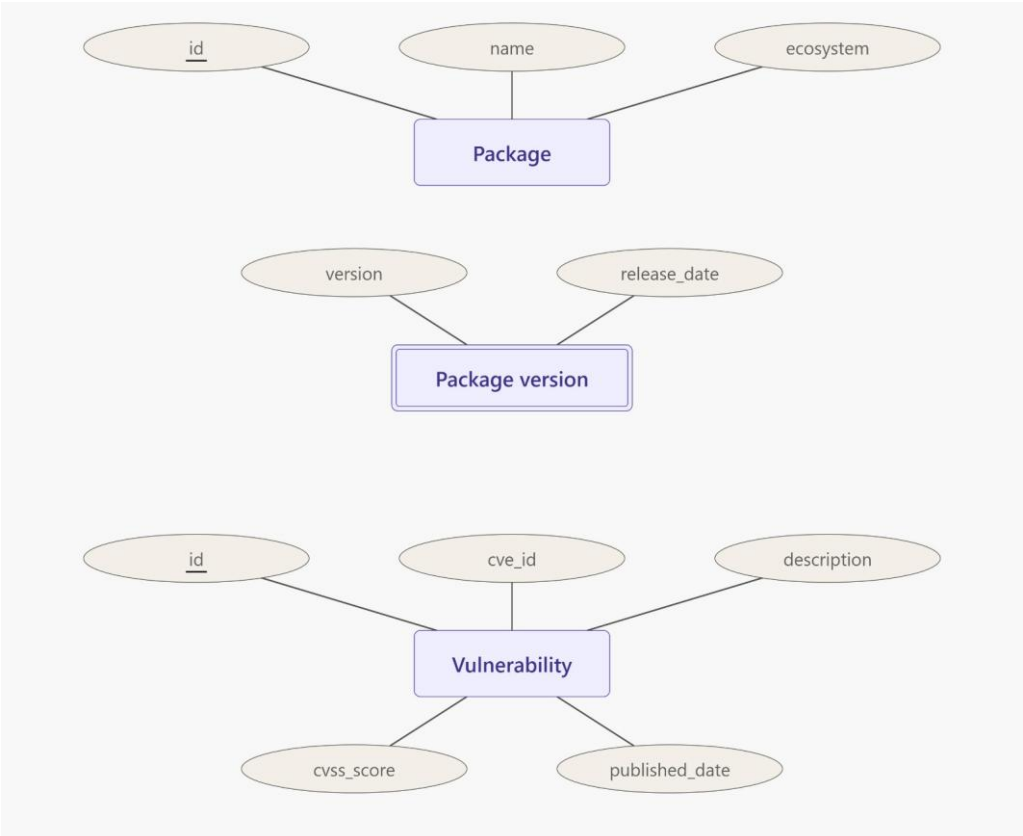


Figure 3.3: ER diagram – attributes of Package, Package version and Vulnerability

4. ER TO RELATIONAL MAPPING

4.1 Steps of algorithm for the chosen problem

Step 1 – Strong entities. User, Project, Package and Vulnerability were mapped to their own tables, with simple attributes as columns and the key attribute (id) as the primary key. cve_id of Vulnerability and email of User carry UNIQUE constraints.

Step 2 – Weak entities. Package version (owner: Package) and Risk snapshot (owner: Project) received the owner's primary key as a foreign key with ON DELETE CASCADE, so deleting a package or project removes its dependent rows.

Step 3 – 1:1 relationships. The model contains no 1:1 relationships.

Step 4 – 1:N relationships. The primary key of the "one" side was placed as a foreign key on the "N" side: User→Project (owner_id), Package→Package version (package_id) and Project→Risk snapshot (project_id).

Step 5 – M:N relationships. Each became a separate table: PROJECT_DEPENDENCIES (Project – Package version, with dependency_type and added_at), VULN_AFFECTED_VERSIONS (Vulnerability – Package version, with version_range) and ALERTS (Project – Vulnerability, with status and created_at). Each has its own surrogate id plus a UNIQUE constraint on the participating keys.

Step 6 – Multivalued attributes. The model contains none.

Step 7 – N-ary relationships. The model contains none; all relationships are binary.

4.2 Relational mapping table

Relational table	Primary key	Important foreign keys
USERS	id	—
PROJECTS	id	owner_id → USERS.id
PACKAGES	id	—
PACKAGE_VERSIONS	id	package_id → PACKAGES.id
PROJECT_DEPENDENCIES	id	project_id → PROJECTS.id; package_version_id → PACKAGE_VERSIONS.id
VULNERABILITIES	id	—
VULN_AFFECTED_VERSIONS	id	vulnerability_id → VULNERABILITIES.id; package_version_id → PACKAGE_VERSIONS.id
ALERTS	id	project_id → PROJECTS.id; vulnerability_id → VULNERABILITIES.id
RISK_SNAPSHOTS	id	project_id → PROJECTS.id

4.3 Complete diagram of relational mapping

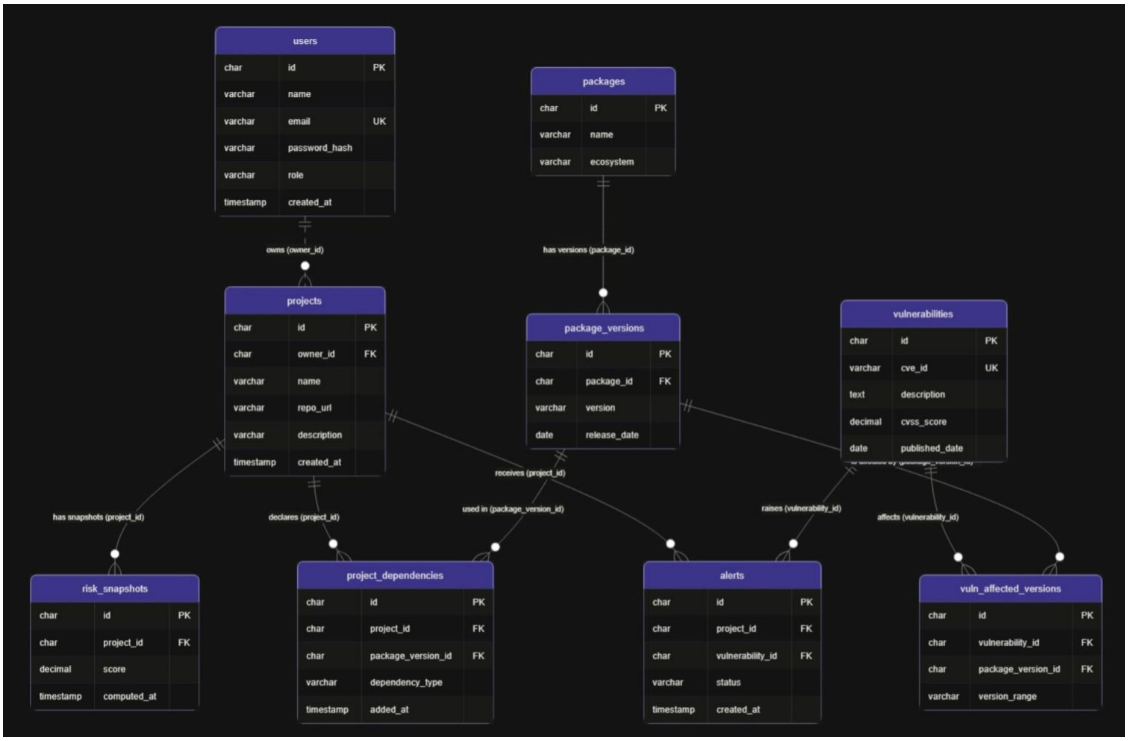


Figure 4.1: Relational schema (primary keys, foreign keys and cardinalities)

5. DDL STATEMENTS

Data Definition Language (DDL) statements define and modify the database structure. The SDVIT schema is created in MySQL using CREATE TABLE statements for the nine tables below. Constraints used include PRIMARY KEY (UUID defaults), FOREIGN KEY with ON DELETE CASCADE, UNIQUE, NOT NULL, DEFAULT, CHECK (for example ecosystem and role domains and $0 \leq$ cvss_score ≤ 10) and indexes on frequently used lookups.

Table	Description
users	User accounts and roles.
projects	Projects owned by users; includes project description.
packages	Package names and ecosystem values.
package_versions	Released versions of packages.
project_dependencies	Project-to-package-version associations.
vulnerabilities	CVE and CVSS vulnerability information.
vuln_affected_versions	Vulnerability-to-affected-version mapping.
alerts	Project vulnerability alert and status records.
risk_snapshots	Historical risk score records.

5.1 CREATE TABLE statements

The complete DDL script (file SDVIT_DDL_ONLY.sql) is reproduced below; the indexes and the ALTER statement follow in Section 5.2. The screenshots after it show the script as run and the resulting schema.

```
DROP DATABASE IF EXISTS sdvitsystem;
CREATE DATABASE sdvitsystem;
USE sdvitsystem;


CREATE TABLE users (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    name            VARCHAR(120) NOT NULL,
    email           VARCHAR(255) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    role            VARCHAR(20)  NOT NULL DEFAULT 'developer'
                        CHECK (role IN ('admin', 'developer')),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;


CREATE TABLE projects (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    owner_id        CHAR(36) NOT NULL,
    name            VARCHAR(150) NOT NULL,
    repo_url        VARCHAR(500),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_owner_project (owner_id, name),
    CONSTRAINT fk_projects_owner FOREIGN KEY (owner_id) REFERENCES users(id)
) ENGINE=InnoDB;


CREATE TABLE packages (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    name            VARCHAR(150) NOT NULL,
    ecosystem       VARCHAR(30)  NOT NULL
                        CHECK (ecosystem IN ('npm', 'pypi', 'maven', 'cargo',
                            'other')),
```

```
    UNIQUE KEY uq_package_name_ecosystem (name, ecosystem)
) ENGINE=InnoDB;

CREATE TABLE package_versions (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    package_id      CHAR(36) NOT NULL,
    version         VARCHAR(50) NOT NULL,
    release_date    DATE,
    UNIQUE KEY uq_package_version (package_id, version),
    CONSTRAINT fk_pkgver_package FOREIGN KEY (package_id)
        REFERENCES packages(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE project_dependencies (
    id                      CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id              CHAR(36) NOT NULL,
    package_version_id      CHAR(36) NOT NULL,
    dependency_type         VARCHAR(20) NOT NULL DEFAULT 'direct'
                                CHECK (dependency_type IN ('direct', 'dev',
                                    'transitive')),
    added_at                TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_project_pkgver (project_id, package_version_id),
    CONSTRAINT fk_pd_project FOREIGN KEY (project_id)
        REFERENCES projects(id) ON DELETE CASCADE,
    CONSTRAINT fk_pd_pkgver FOREIGN KEY (package_version_id)
        REFERENCES package_versions(id)
) ENGINE=InnoDB;

CREATE TABLE vulnerabilities (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    cve_id          VARCHAR(30) NOT NULL UNIQUE,
    description     TEXT NOT NULL,
    cvss_score      DECIMAL(3,1) CHECK (cvss_score >= 0 AND cvss_score <= 10),
    published_date  DATE NOT NULL
) ENGINE=InnoDB;

CREATE TABLE vuln_affected_versions (
    id                      CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    vulnerability_id        CHAR(36) NOT NULL,
    package_version_id      CHAR(36) NOT NULL,
    version_range           VARCHAR(100),
    UNIQUE KEY uq_vuln_pkgver (vulnerability_id, package_version_id),
    CONSTRAINT fk_vav_vuln FOREIGN KEY (vulnerability_id)
        REFERENCES vulnerabilities(id) ON DELETE CASCADE,
    CONSTRAINT fk_vav_pkgver FOREIGN KEY (package_version_id)
        REFERENCES package_versions(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE alerts (
    id                  CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id          CHAR(36) NOT NULL,
    vulnerability_id    CHAR(36) NOT NULL,
    status              VARCHAR(20) NOT NULL DEFAULT 'open'
                            CHECK (status IN ('open', 'acknowledged', 'resolved')),
    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_project_vuln (project_id, vulnerability_id),
    CONSTRAINT fk_alerts_project FOREIGN KEY (project_id)
        REFERENCES projects(id) ON DELETE CASCADE,
    CONSTRAINT fk_alerts_vuln FOREIGN KEY (vulnerability_id)
        REFERENCES vulnerabilities(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE risk_snapshots (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id      CHAR(36) NOT NULL,
    score           DECIMAL(5,2) NOT NULL,
    computed_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_risk_project FOREIGN KEY (project_id)
        REFERENCES projects(id) ON DELETE CASCADE
```

```
) ENGINE=InnoDB;
```

```sql
-- ========================================================================
-- Software Dependency Vulnerability & Impact Tracking System
-- MySQL 8.x DDL (SQL side; Neo4j graph layer stores DEPENDS_ON edges
-- separately for transitive traversal)
-- ========================================================================

-- IDs are stored as CHAR(36) UUID strings (generated with UUID() in
-- application code or via DEFAULT (UUID()) on MySQL 8.0.13+)

-- ------------------------------------------------------------------------
-- USERS  (auth + role-based access: Admin / Developer)
-- ------------------------------------------------------------------------
DROP DATABASE IF EXISTS sdvitsystem;
CREATE DATABASE sdvitsystem;
USE sdvitsystem;

CREATE TABLE users (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    name            VARCHAR(120) NOT NULL,
    email           VARCHAR(255) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    role            VARCHAR(20)  NOT NULL DEFAULT 'developer'
                        CHECK (role IN ('admin', 'developer')),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- PROJECTS  (each owned by a user; the thing whose exposure we track)
-- ------------------------------------------------------------------------
CREATE TABLE projects (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    owner_id        CHAR(36) NOT NULL,
    name            VARCHAR(150) NOT NULL,
    repo_url        VARCHAR(500),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_owner_project (owner_id, name),
    CONSTRAINT fk_projects_owner FOREIGN KEY (owner_id) REFERENCES users(id)
) ENGINE=InnoDB;

```

DDL script (part 1)

```sql
-- ------------------------------------------------------------------------
-- PACKAGES  (distinct library/package names, per ecosystem)
-- ------------------------------------------------------------------------
CREATE TABLE packages (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    name            VARCHAR(150) NOT NULL,
    ecosystem       VARCHAR(30)  NOT NULL
                        CHECK (ecosystem IN ('npm', 'pypi', 'maven', 'cargo', 'other')),
    UNIQUE KEY uq_package_name_ecosystem (name, ecosystem)
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- PACKAGE_VERSIONS  (specific released versions of a package)
-- ------------------------------------------------------------------------
CREATE TABLE package_versions (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    package_id      CHAR(36) NOT NULL,
    version         VARCHAR(50) NOT NULL,
    release_date    DATE,
    UNIQUE KEY uq_package_version (package_id, version),
    CONSTRAINT fk_pkgver_package FOREIGN KEY (package_id) REFERENCES packages(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- PROJECT_DEPENDENCIES  (direct dependency declarations; transitive
-- resolution itself lives in the Neo4j graph, not here)
-- ------------------------------------------------------------------------
CREATE TABLE project_dependencies (
    id                  CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id          CHAR(36) NOT NULL,
    package_version_id  CHAR(36) NOT NULL,
    dependency_type     VARCHAR(20) NOT NULL DEFAULT 'direct'
                            CHECK (dependency_type IN ('direct', 'dev', 'transitive')),
    added_at            TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_project_pkgver (project_id, package_version_id),
    CONSTRAINT fk_pd_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE,
    CONSTRAINT fk_pd_pkgver  FOREIGN KEY (package_version_id) REFERENCES package_versions(id)
) ENGINE=InnoDB;
```

DDL script (part 2)

```sql
-- ------------------------------------------------------------------------
-- VULNERABILITIES  (CVE records, sourced from NVD/OSV feeds)
-- ------------------------------------------------------------------------
CREATE TABLE vulnerabilities (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    cve_id          VARCHAR(30) NOT NULL UNIQUE,
    description     TEXT NOT NULL,
    cvss_score      DECIMAL(3,1) CHECK (cvss_score >= 0 AND cvss_score <= 10),
    published_date  DATE NOT NULL
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- VULN_AFFECTED_VERSIONS  (which package versions a CVE applies to)
-- ------------------------------------------------------------------------
CREATE TABLE vuln_affected_versions (
    id                    CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    vulnerability_id      CHAR(36) NOT NULL,
    package_version_id    CHAR(36) NOT NULL,
    version_range         VARCHAR(100),
    UNIQUE KEY uq_vuln_pkgver (vulnerability_id, package_version_id),
    CONSTRAINT fk_vav_vuln   FOREIGN KEY (vulnerability_id) REFERENCES vulnerabilities(id) ON DELETE CASCADE,
    CONSTRAINT fk_vav_pkgver FOREIGN KEY (package_version_id) REFERENCES package_versions(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- ALERTS  (auto-generated when a CVE impacts a project; see trigger)
-- ------------------------------------------------------------------------
CREATE TABLE alerts (
    id                CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id        CHAR(36) NOT NULL,
    vulnerability_id  CHAR(36) NOT NULL,
    status            VARCHAR(20) NOT NULL DEFAULT 'open'
                          CHECK (status IN ('open', 'acknowledged', 'resolved')),
    created_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_project_vuln (project_id, vulnerability_id),
    CONSTRAINT fk_alerts_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE,
    CONSTRAINT fk_alerts_vuln    FOREIGN KEY (vulnerability_id) REFERENCES vulnerabilities(id) ON DELETE CASCADE
) ENGINE=InnoDB;

```

DDL script (part 3)

```sql
-- ------------------------------------------------------------------------
-- RISK_SNAPSHOTS  (historical aggregate risk score per project)
-- ------------------------------------------------------------------------
CREATE TABLE risk_snapshots (
    id              CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    project_id      CHAR(36) NOT NULL,
    score           DECIMAL(5,2) NOT NULL,
    computed_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_risk_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ========================================================================
-- INDEXES (beyond the automatic ones on PK / UNIQUE constraints)
-- ========================================================================
CREATE INDEX idx_package_versions_package_id ON package_versions(package_id);
CREATE INDEX idx_project_dependencies_project_id ON project_dependencies(project_id);
CREATE INDEX idx_project_dependencies_pkg_version_id ON project_dependencies(package_version_id);
CREATE INDEX idx_vuln_affected_versions_vuln_id ON vuln_affected_versions(vulnerability_id);
CREATE INDEX idx_vuln_affected_versions_pkg_version_id ON vuln_affected_versions(package_version_id);
CREATE INDEX idx_alerts_project_id ON alerts(project_id);
CREATE INDEX idx_alerts_status ON alerts(status);
CREATE INDEX idx_risk_snapshots_project_id ON risk_snapshots(project_id);
CREATE INDEX idx_vulnerabilities_cvss_score ON vulnerabilities(cvss_score);

```

DDL script (part 4) – risk_snapshots and indexes

```
mysql> show tables;
+-------------------------+
| Tables_in_sdvitsystem   |
+-------------------------+
| alerts                  |
| package_versions        |
| packages                |
| project_dependencies    |
| project_risk_overview   |
| projects                |
| risk_snapshots          |
| users                   |
| vuln_affected_versions  |
| vulnerabilities         |
+-------------------------+
```

Output of SHOW TABLES after the DDL script (9 base tables and the project_risk_overview view)

```
mysql> desc alerts;
+------------------+-------------+------+-----+-------------------+-------------------+
| Field            | Type        | Null | Key | Default           | Extra             |
+------------------+-------------+------+-----+-------------------+-------------------+
| id               | char(36)    | NO   | PRI | uuid()            | DEFAULT_GENERATED |
| project_id       | char(36)    | NO   | MUL | NULL              |                   |
| vulnerability_id | char(36)    | NO   | MUL | NULL              |                   |
| status           | varchar(20) | NO   | MUL | open              |                   |
| created_at       | timestamp   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+-------------+------+-----+-------------------+-------------------+
5 rows in set (0.00 sec)

mysql> desc package_versions;
+--------------+-------------+------+-----+---------+-------------------+
| Field        | Type        | Null | Key | Default | Extra             |
+--------------+-------------+------+-----+---------+-------------------+
| id           | char(36)    | NO   | PRI | uuid()  | DEFAULT_GENERATED |
| package_id   | char(36)    | NO   | MUL | NULL    |                   |
| version      | varchar(50) | NO   |     | NULL    |                   |
| release_date | date        | YES  |     | NULL    |                   |
+--------------+-------------+------+-----+---------+-------------------+
4 rows in set (0.00 sec)

mysql> desc packages;
+-----------+--------------+------+-----+---------+-------------------+
| Field     | Type         | Null | Key | Default | Extra             |
+-----------+--------------+------+-----+---------+-------------------+
| id        | char(36)     | NO   | PRI | uuid()  | DEFAULT_GENERATED |
| name      | varchar(150) | NO   | MUL | NULL    |                   |
| ecosystem | varchar(30)  | NO   |     | NULL    |                   |
+-----------+--------------+------+-----+---------+-------------------+
3 rows in set (0.00 sec)
```

DESC alerts, package_versions, packages

```
mysql> desc project_dependencies;
+--------------------+-------------+------+-----+-------------------+-------------------+
| Field              | Type        | Null | Key | Default           | Extra             |
+--------------------+-------------+------+-----+-------------------+-------------------+
| id                 | char(36)    | NO   | PRI | uuid()            | DEFAULT_GENERATED |
| project_id         | char(36)    | NO   | MUL | NULL              |                   |
| package_version_id | char(36)    | NO   | MUL | NULL              |                   |
| dependency_type    | varchar(20) | NO   |     | direct            |                   |
| added_at           | timestamp   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------------+-------------+------+-----+-------------------+-------------------+
5 rows in set (0.00 sec)

mysql> desc projects;
+------------+--------------+------+-----+-------------------+-------------------+
| Field      | Type         | Null | Key | Default           | Extra             |
+------------+--------------+------+-----+-------------------+-------------------+
| id         | char(36)     | NO   | PRI | uuid()            | DEFAULT_GENERATED |
| owner_id   | char(36)     | NO   | MUL | NULL              |                   |
| name       | varchar(150) | NO   |     | NULL              |                   |
| repo_url   | varchar(500) | YES  |     | NULL              |                   |
| created_at | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------+--------------+------+-----+-------------------+-------------------+
5 rows in set (0.00 sec)

mysql> desc risk_snapshots;
+-------------+--------------+------+-----+-------------------+-------------------+
| Field       | Type         | Null | Key | Default           | Extra             |
+-------------+--------------+------+-----+-------------------+-------------------+
| id          | char(36)     | NO   | PRI | uuid()            | DEFAULT_GENERATED |
| project_id  | char(36)     | NO   | MUL | NULL              |                   |
| score       | decimal(5,2) | NO   |     | NULL              |                   |
| computed_at | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+--------------+------+-----+-------------------+-------------------+
```

DESC project_dependencies, projects, risk_snapshots

```
mysql> desc users;
+---------------+--------------+------+-----+-------------------+-------------------+
| Field         | Type         | Null | Key | Default           | Extra             |
+---------------+--------------+------+-----+-------------------+-------------------+
| id            | char(36)     | NO   | PRI | uuid()            | DEFAULT_GENERATED |
| name          | varchar(120) | NO   |     | NULL              |                   |
| email         | varchar(255) | NO   | UNI | NULL              |                   |
| password_hash | varchar(255) | NO   |     | NULL              |                   |
| role          | varchar(20)  | NO   |     | developer         |                   |
| created_at    | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+--------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)

mysql> desc vuln_affected_versions;
+--------------------+--------------+------+-----+---------+-------------------+
| Field              | Type         | Null | Key | Default | Extra             |
+--------------------+--------------+------+-----+---------+-------------------+
| id                 | char(36)     | NO   | PRI | uuid()  | DEFAULT_GENERATED |
| vulnerability_id   | char(36)     | NO   | MUL | NULL    |                   |
| package_version_id | char(36)     | NO   | MUL | NULL    |                   |
| version_range      | varchar(100) | YES  |     | NULL    |                   |
+--------------------+--------------+------+-----+---------+-------------------+
4 rows in set (0.00 sec)

mysql> desc vulnerabilities;
+----------------+--------------+------+-----+---------+-------------------+
| Field          | Type         | Null | Key | Default | Extra             |
+----------------+--------------+------+-----+---------+-------------------+
| id             | char(36)     | NO   | PRI | uuid()  | DEFAULT_GENERATED |
| cve_id         | varchar(30)  | NO   | UNI | NULL    |                   |
| description    | text         | NO   |     | NULL    |                   |
| cvss_score     | decimal(3,1) | YES  | MUL | NULL    |                   |
| published_date | date         | NO   |     | NULL    |                   |
+----------------+--------------+------+-----+---------+-------------------+
```

DESC users, vuln_affected_versions, vulnerabilities

5.2 ALTER, INDEX and DROP statements

```
ALTER TABLE projects ADD COLUMN description VARCHAR(500);


CREATE INDEX idx_package_versions_package_id ON package_versions(package_id);
CREATE INDEX idx_project_dependencies_project_id ON project_dependencies(project_id);
CREATE INDEX idx_project_dependencies_pkg_version_id ON
project_dependencies(package_version_id);
CREATE INDEX idx_vuln_affected_versions_vuln_id ON
vuln_affected_versions(vulnerability_id);
CREATE INDEX idx_vuln_affected_versions_pkg_version_id ON
vuln_affected_versions(package_version_id);
CREATE INDEX idx_alerts_project_id ON alerts(project_id);
CREATE INDEX idx_alerts_status ON alerts(status);
CREATE INDEX idx_risk_snapshots_project_id ON risk_snapshots(project_id);
CREATE INDEX idx_vulnerabilities_cvss_score ON vulnerabilities(cvss_score);


DROP DATABASE IF EXISTS sdvitsystem;   -- used at the top of the DDL script
DROP VIEW IF EXISTS project_risk_overview;
```

6. DML STATEMENTS

Data Manipulation Language (DML) statements insert, modify and delete records stored in database tables. Sample data is inserted into the SDVIT tables to demonstrate users, projects, packages, package versions, dependencies, vulnerabilities, affected versions, alerts and risk snapshots. The inserts were executed in dependency order so that all foreign keys are satisfied.

Representative INSERT statements from the sample-data script are shown below. Project dependencies are inserted before the CVE mappings so that the trigger on vuln_affected_versions can find the projects that already use each affected version.

```
INSERT INTO users (id, name, email, password_hash, role) VALUES
('a0000000-0000-0000-0000-000000000001', 'Admin User', 'admin@sdvit.example',
    'hashed_pw_placeholder', 'admin'),
('a0000000-0000-0000-0000-000000000002', 'Samaresh',
    'samaresh@sdvit.example','hashed_pw_placeholder', 'developer'),
('a0000000-0000-0000-0000-000000000003', 'Asha Rao', 'asha@sdvit.example',
    'hashed_pw_placeholder', 'developer');


INSERT INTO packages (id, name, ecosystem) VALUES
('c0000000-0000-0000-0000-000000000001', 'lodash',     'npm'),
('c0000000-0000-0000-0000-000000000002', 'express',    'npm'),
('c0000000-0000-0000-0000-000000000003', 'requests',   'pypi'),
('c0000000-0000-0000-0000-000000000004', 'log4j-core', 'maven'),
('c0000000-0000-0000-0000-000000000005', 'django',     'pypi'),
('c0000000-0000-0000-0000-000000000006', 'serde',      'cargo');


INSERT INTO vulnerabilities (id, cve_id, description, cvss_score,
    published_date) VALUES
('f0000000-0000-0000-0000-000000000001', 'CVE-2021-44228',
    'Log4Shell: remote code execution via JNDI lookups in log4j-core', 10.0,
        '2021-12-10'),
('f0000000-0000-0000-0000-000000000002', 'CVE-2020-8203',
    'lodash prototype pollution in zipObjectDeep', 7.4, '2020-07-15'),
('f0000000-0000-0000-0000-000000000003', 'CVE-2018-18074',
    'requests sends Authorization header to a different host on redirect', 9.8,
        '2018-10-09'),
('f0000000-0000-0000-0000-000000000004', 'CVE-2024-29041',
    'express open redirect via malformed URLs', 6.1, '2024-03-25'),
('f0000000-0000-0000-0000-000000000005', 'CVE-2021-35042',
    'Django SQL injection through QuerySet.order_by', 9.8, '2021-07-02');


INSERT INTO vuln_affected_versions (vulnerability_id, package_version_id,
    version_range) VALUES
('f0000000-0000-0000-0000-000000000001', 'd0000000-0000-0000-0000-000000000007',
    '>=2.0-beta9 <2.15.0'),
('f0000000-0000-0000-0000-000000000002', 'd0000000-0000-0000-0000-000000000001',
    '<4.17.19'),
('f0000000-0000-0000-0000-000000000003', 'd0000000-0000-0000-0000-000000000005',
    '<2.20.0'),
('f0000000-0000-0000-0000-000000000004', 'd0000000-0000-0000-0000-000000000003',
    '<4.19.2'),
('f0000000-0000-0000-0000-000000000004', 'd0000000-0000-0000-0000-000000000004',
    '<4.19.2'),
('f0000000-0000-0000-0000-000000000005', 'd0000000-0000-0000-0000-000000000009',
    '>=3.2 <3.2.5');
```

6.1 Rows loaded per table

Table	Rows
users	3

Table	Rows
projects	3
packages	6
package_versions	10
project_dependencies	10
vulnerabilities	5
vuln_affected_versions	6
alerts	5
risk_snapshots	5

Counts are the rows present after the sample-data script and its demonstrations have run; alerts and risk_snapshots rows are created by triggers and procedures rather than by direct INSERTs. Run SELECT COUNT(*) FROM <table>; to check the live database, which also contains anything added through the web application.

6.2 Sample data



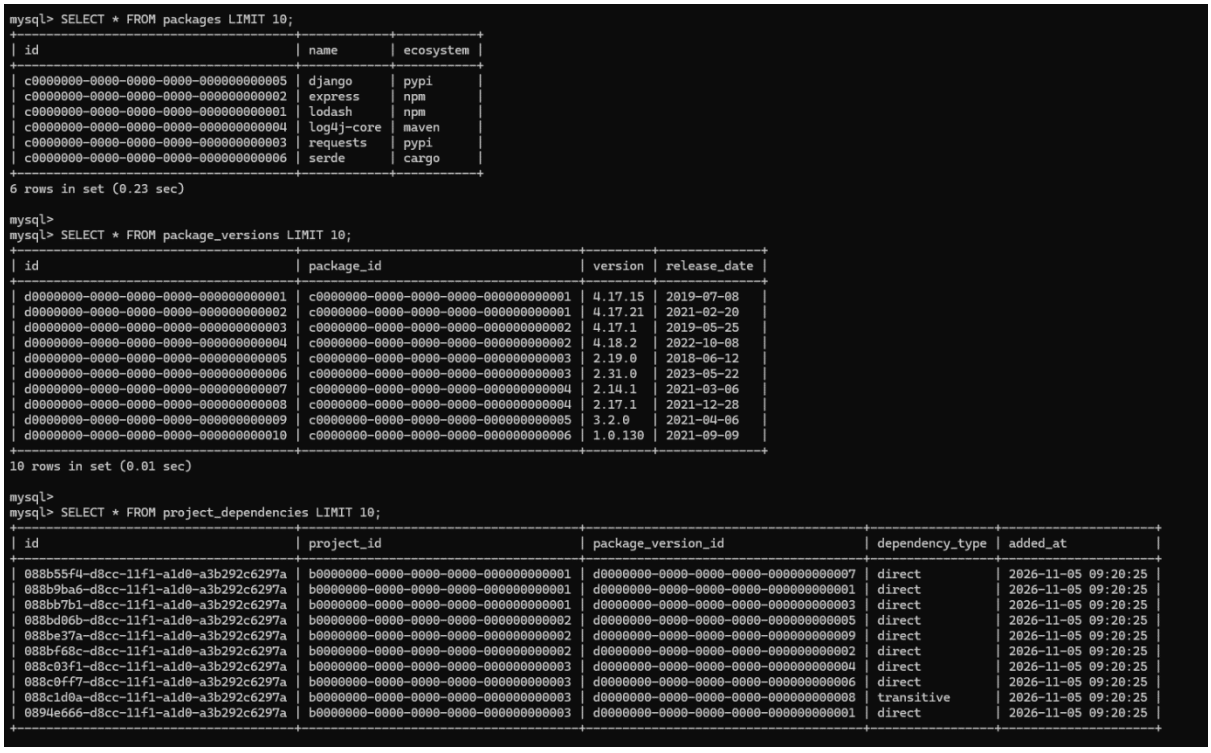
*SELECT * FROM users; SELECT * FROM projects;*

```
mysql> SELECT status, COUNT(*) AS alerts
    -> FROM alerts
    -> GROUP BY status;
+--------------+--------+
| status       | alerts |
+--------------+--------+
| acknowledged |      1 |
| open         |      3 |
| resolved     |      1 |
+--------------+--------+
3 rows in set (0.00 sec)

mysql> SELECT COUNT(*) AS leftover_versions
    -> FROM package_versions
    -> WHERE package_id =
    -> 'c0000000-0000-0000-0000-000000000099';
+-------------------+
| leftover_versions |
+-------------------+
|                 0 |
+-------------------+
1 row in set (0.00 sec)
```

Alert counts by status

Verification of Data Manipulation Operations (DML):



```
mysql> SELECT * FROM packages LIMIT 10;
+--------------------------------------+-----------+------------+
| id                                   | name      | ecosystem  |
+--------------------------------------+-----------+------------+
| c0000000-0000-0000-0000-000000000005 | django    | pypi       |
| c0000000-0000-0000-0000-000000000002 | express   | npm        |
| c0000000-0000-0000-0000-000000000001 | lodash    | npm        |
| c0000000-0000-0000-0000-000000000004 | log4j-core| maven      |
| c0000000-0000-0000-0000-000000000003 | requests  | pypi       |
| c0000000-0000-0000-0000-000000000006 | serde     | cargo      |
+--------------------------------------+-----------+------------+
6 rows in set (0.23 sec)

mysql>
mysql> SELECT * FROM package_versions LIMIT 10;
+--------------------------------------+--------------------------------------+---------+--------------+
| id                                   | package_id                           | version | release_date |
+--------------------------------------+--------------------------------------+---------+--------------+
| d0000000-0000-0000-0000-000000000001 | c0000000-0000-0000-0000-000000000001 | 4.17.15 | 2019-07-08   |
| d0000000-0000-0000-0000-000000000002 | c0000000-0000-0000-0000-000000000001 | 4.17.21 | 2021-02-20   |
| d0000000-0000-0000-0000-000000000003 | c0000000-0000-0000-0000-000000000002 | 4.17.1  | 2019-05-25   |
| d0000000-0000-0000-0000-000000000004 | c0000000-0000-0000-0000-000000000002 | 4.18.2  | 2022-10-08   |
| d0000000-0000-0000-0000-000000000005 | c0000000-0000-0000-0000-000000000003 | 2.19.0  | 2018-06-12   |
| d0000000-0000-0000-0000-000000000006 | c0000000-0000-0000-0000-000000000003 | 2.31.0  | 2023-05-22   |
| d0000000-0000-0000-0000-000000000007 | c0000000-0000-0000-0000-000000000004 | 2.14.1  | 2021-03-06   |
| d0000000-0000-0000-0000-000000000008 | c0000000-0000-0000-0000-000000000004 | 2.17.1  | 2021-12-28   |
| d0000000-0000-0000-0000-000000000009 | c0000000-0000-0000-0000-000000000005 | 3.2.0   | 2021-04-06   |
| d0000000-0000-0000-0000-000000000010 | c0000000-0000-0000-0000-000000000006 | 1.0.130 | 2021-09-09   |
+--------------------------------------+--------------------------------------+---------+--------------+
10 rows in set (0.01 sec)

mysql>
mysql> SELECT * FROM project_dependencies LIMIT 10;
+--------------------------------------+--------------------------------------+--------------------------------------+-----------------+---------------------+
| id                                   | project_id                           | package_version_id                   | dependency_type | added_at            |
+--------------------------------------+--------------------------------------+--------------------------------------+-----------------+---------------------+
| 088b55f4-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | d0000000-0000-0000-0000-000000000007 | direct          | 2026-11-05 09:20:25 |
| 088b9ba6-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | d0000000-0000-0000-0000-000000000001 | direct          | 2026-11-05 09:20:25 |
| 088bb7b1-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | d0000000-0000-0000-0000-000000000003 | direct          | 2026-11-05 09:20:25 |
| 088bd06b-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | d0000000-0000-0000-0000-000000000005 | direct          | 2026-11-05 09:20:25 |
| 088be37a-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | d0000000-0000-0000-0000-000000000009 | direct          | 2026-11-05 09:20:25 |
| 088bf68c-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | d0000000-0000-0000-0000-000000000002 | direct          | 2026-11-05 09:20:25 |
| 088c03f1-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 | d0000000-0000-0000-0000-000000000004 | direct          | 2026-11-05 09:20:25 |
| 088c0ff7-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 | d0000000-0000-0000-0000-000000000006 | direct          | 2026-11-05 09:20:25 |
| 088c1d0a-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 | d0000000-0000-0000-0000-000000000008 | transitive      | 2026-11-05 09:20:25 |
| 0894e666-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 | d0000000-0000-0000-0000-000000000001 | direct          | 2026-11-05 09:20:25 |
+--------------------------------------+--------------------------------------+--------------------------------------+-----------------+---------------------+
```

```
mysql> SELECT * FROM vulnerabilities LIMIT 10;
+--------------------------------------+---------------+-----------------------------------------------------------------------+------------+----------------+
| id                                   | cve_id        | description                                                           | cvss_score | published_date |
+--------------------------------------+---------------+-----------------------------------------------------------------------+------------+----------------+
| f0000000-0000-0000-0000-000000000001 | CVE-2021-44228 | Log4Shell: remote code execution via JNDI lookups in log4j-core      |       10.0 | 2021-12-10     |
| f0000000-0000-0000-0000-000000000002 | CVE-2020-8203  | lodash prototype pollution in zipObjectDeep                           |        7.4 | 2020-07-15     |
| f0000000-0000-0000-0000-000000000003 | CVE-2018-18074 | requests sends Authorization header to a different host on redirect   |        9.8 | 2018-10-09     |
| f0000000-0000-0000-0000-000000000004 | CVE-2024-29041 | express open redirect via malformed URLs                              |        6.1 | 2024-03-25     |
| f0000000-0000-0000-0000-000000000005 | CVE-2021-35042 | Django SQL injection through QuerySet.order_by                        |        9.8 | 2021-07-02     |
+--------------------------------------+---------------+-----------------------------------------------------------------------+------------+----------------+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM vuln_affected_versions LIMIT 10;
+--------------------------------------+--------------------------------------+--------------------------------------+-------------------+
| id                                   | vulnerability_id                     | package_version_id                   | version_range     |
+--------------------------------------+--------------------------------------+--------------------------------------+-------------------+
| 088dcb26-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000001 | d0000000-0000-0000-0000-000000000007 | >=2.0-beta9 <2.15.0 |
| 088de54b-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000002 | d0000000-0000-0000-0000-000000000001 | <4.17.19          |
| 088def56-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000003 | d0000000-0000-0000-0000-000000000005 | <2.20.0           |
| 088df5bf-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000004 | d0000000-0000-0000-0000-000000000003 | <4.19.2           |
| 088df86f-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000004 | d0000000-0000-0000-0000-000000000004 | <4.19.2           |
| 088dfb08-d8cc-11f1-a1d0-a3b292c6297a | f0000000-0000-0000-0000-000000000005 | d0000000-0000-0000-0000-000000000009 | >=3.2 <3.2.5      |
+--------------------------------------+--------------------------------------+--------------------------------------+-------------------+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM alerts LIMIT 10;
+--------------------------------------+--------------------------------------+--------------------------------------+--------------+---------------------+
| id                                   | project_id                           | vulnerability_id                     | status       | created_at          |
+--------------------------------------+--------------------------------------+--------------------------------------+--------------+---------------------+
| 088ddfd1-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | f0000000-0000-0000-0000-000000000001 | open         | 2026-11-05 09:20:25 |
| 088decfc-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | f0000000-0000-0000-0000-000000000002 | resolved     | 2026-11-05 09:20:25 |
| 088df3a1-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | f0000000-0000-0000-0000-000000000003 | acknowledged | 2026-11-05 09:20:25 |
| 088dfee6-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | f0000000-0000-0000-0000-000000000005 | open         | 2026-11-05 09:20:25 |
| 0894fe33-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 | f0000000-0000-0000-0000-000000000002 | open         | 2026-11-05 09:20:25 |
+--------------------------------------+--------------------------------------+--------------------------------------+--------------+---------------------+
5 rows in set (0.00 sec)
```

```
mysql>
mysql> SELECT * FROM risk_snapshots LIMIT 10;
+--------------------------------------+--------------------------------------+-------+---------------------+
| id                                   | project_id                           | score | computed_at         |
+--------------------------------------+--------------------------------------+-------+---------------------+
| 08988c24-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000002 | 19.60 | 2026-11-05 09:20:25 |
| 08991566-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | 17.40 | 2026-11-05 09:20:25 |
| 0899abec-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000003 |  7.40 | 2026-11-05 09:20:25 |
| 089cb89f-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | 17.40 | 2026-11-05 09:20:25 |
| 0937ce52-d8cc-11f1-a1d0-a3b292c6297a | b0000000-0000-0000-0000-000000000001 | 10.00 | 2026-11-05 09:20:26 |
+--------------------------------------+--------------------------------------+-------+---------------------+
5 rows in set (0.00 sec)
```

7. QUERIES

SQL queries are used to retrieve and analyse information stored in SDVIT. The examples below demonstrate aggregate queries, update and delete operations, correlated subqueries and nested subqueries.

7.1 Simple query with GROUP BY, aggregate

Projects with at least one open high-severity alert (CVSS ≥ 7.0), with the number of such alerts (multi-table JOIN, WHERE, GROUP BY, HAVING, aggregate COUNT, ORDER BY).

```
SELECT p.name, COUNT(a.id) AS open_high_alerts
FROM projects p
JOIN alerts a ON a.project_id = p.id
JOIN vulnerabilities v ON v.id = a.vulnerability_id
WHERE a.status = 'open' AND v.cvss_score >= 7.0
GROUP BY p.name
HAVING COUNT(a.id) >= 1
ORDER BY open_high_alerts DESC;
```

```
mysql> SELECT
    ->     p.name,
    ->     COUNT(a.id) AS open_high_alerts
    -> FROM projects p
    -> JOIN alerts a ON a.project_id = p.id
    -> JOIN vulnerabilities v ON v.id = a.vulnerability_id
    -> WHERE a.status = 'open'
    ->   AND v.cvss_score >= 7.0
    -> GROUP BY p.name
    -> HAVING COUNT(a.id) >= 1
    -> ORDER BY open_high_alerts DESC;
+-----------------+------------------+
| name            | open_high_alerts |
+-----------------+------------------+
| payment-service |                1 |
| analytics-api   |                1 |
| mobile-backend  |                1 |
+-----------------+------------------+
3 rows in set (0.00 sec)
```

Output of query 7.1

7.2 UPDATE operation

An UPDATE query changes existing values without deleting the record. The following example marks a selected alert as resolved.

```
UPDATE alerts
SET status = 'resolved'
WHERE project_id = 'b0000000-0000-0000-0000-000000000001'
  AND vulnerability_id = 'f0000000-0000-0000-0000-000000000001';
```

Because the status changes from open to resolved, the UPDATE also fires trg_risk_snapshot_on_alert_resolved, which records a new risk snapshot for the project (Section 8.2).

```
mysql> UPDATE alerts
    -> SET status = 'resolved'
    -> WHERE project_id = 'b0000000-0000-0000-0000-000000000001'
    ->   AND vulnerability_id = 'f0000000-0000-0000-0000-000000000001';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

UPDATE alerts – alert marked resolved

7.3 DELETE operation

A temporary package and one version are inserted and then deleted; the dependent package_versions row is removed automatically through ON DELETE CASCADE.

```
INSERT INTO packages (id, name, ecosystem)
VALUES ('c0000000-0000-0000-0000-000000000099', 'temp-pkg', 'other');

INSERT INTO package_versions (package_id, version)
VALUES ('c0000000-0000-0000-0000-000000000099', '0.0.1');

DELETE FROM packages
WHERE id = 'c0000000-0000-0000-0000-000000000099';

SELECT * FROM package_versions
WHERE package_id = 'c0000000-0000-0000-0000-000000000099';   -- empty set
```

```
mysql> INSERT INTO packages (id, name, ecosystem)
    -> VALUES ('c0000000-0000-0000-0000-000000000099', 'temp-pkg', 'other');
Query OK, 1 row affected (0.03 sec)

mysql>
mysql> INSERT INTO package_versions (package_id, version)
    -> VALUES ('c0000000-0000-0000-0000-000000000099', '0.0.1');
Query OK, 1 row affected (0.04 sec)

mysql> SELECT *
    -> FROM packages
    -> WHERE id = 'c0000000-0000-0000-0000-000000000099';
+--------------------------------------+----------+-----------+
| id                                   | name     | ecosystem |
+--------------------------------------+----------+-----------+
| c0000000-0000-0000-0000-000000000099 | temp-pkg | other     |
+--------------------------------------+----------+-----------+
1 row in set (0.00 sec)

mysql>
mysql> SELECT *
    -> FROM package_versions
    -> WHERE package_id = 'c0000000-0000-0000-0000-000000000099';DELETE FROM packages
+--------------------------------------+--------------------------------------+---------+--------------+
| id                                   | package_id                           | version | release_date |
+--------------------------------------+--------------------------------------+---------+--------------+
| 2a864f01-d68a-11f1-98ac-34aab796cb35 | c0000000-0000-0000-0000-000000000099 | 0.0.1   | NULL         |
+--------------------------------------+--------------------------------------+---------+--------------+
1 row in set (0.00 sec)

    -> WHERE id = 'c0000000-0000-0000-0000-000000000099';
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
    -> FROM packages
    -> WHERE id = 'c0000000-0000-0000-0000-000000000099';
Empty set (0.00 sec)

mysql>
mysql> SELECT *
    -> FROM package_versions
    -> WHERE package_id = 'c0000000-0000-0000-0000-000000000099';
Empty set (0.00 sec)

mysql>
```

DELETE with cascade – package_versions row removed with its package

7.4 Correlated query

Package versions that have no affected-version record, i.e. versions not known to be vulnerable. The inner query refers to pv.id from the outer query, so it is evaluated for each outer row.

```
SELECT pkg.name, pv.version
FROM package_versions pv
JOIN packages pkg ON pkg.id = pv.package_id
WHERE NOT EXISTS (
    SELECT 1 FROM vuln_affected_versions vav
    WHERE vav.package_version_id = pv.id
)
ORDER BY pkg.name, pv.version;
```

```
mysql> SELECT pkg.name, pv.version
    -> FROM package_versions pv
    -> JOIN packages pkg ON pkg.id = pv.package_id
    -> WHERE NOT EXISTS (
    ->     SELECT 1
    ->     FROM vuln_affected_versions vav
    ->     WHERE vav.package_version_id = pv.id
    -> )
    -> ORDER BY pkg.name, pv.version;
+------------+---------+
| name       | version |
+------------+---------+
| lodash     | 4.17.21 |
| log4j-core | 2.17.1  |
| requests   | 2.31.0  |
| serde      | 1.0.130 |
+------------+---------+
4 rows in set (0.00 sec)
```

Output of query 7.4

7.5 Nested query

Vulnerabilities whose CVSS score is above the average CVSS score. The subquery calculates the average once and the outer query filters on it.

```
SELECT cve_id, cvss_score
FROM vulnerabilities
WHERE cvss_score > (SELECT AVG(cvss_score) FROM vulnerabilities)
ORDER BY cvss_score DESC;
```

```
mysql> SELECT cve_id, cvss_score
    -> FROM vulnerabilities
    -> WHERE cvss_score > (
    ->     SELECT AVG(cvss_score)
    ->     FROM vulnerabilities
    -> )
    -> ORDER BY cvss_score DESC;
+----------------+------------+
| cve_id         | cvss_score |
+----------------+------------+
| CVE-2021-44228 |       10.0 |
| CVE-2021-35042 |        9.8 |
| CVE-2018-18074 |        9.8 |
+----------------+------------+
3 rows in set (0.00 sec)
```

Output of query 7.5

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

Stored procedures, functions and triggers are programmable database objects used to encapsulate operations and automate database actions. Procedures are explicitly called to perform a task, functions return a value, and triggers execute automatically when a specified table event occurs.

8.1 Stored procedures or functions

Stored procedures

Procedure	Purpose
recompute_project_risk	Calculates the sum of distinct CVSS scores for a project's open alerts and inserts a risk snapshot.
get_project_vulnerability_report	Returns vulnerability and alert details for a project, including severity.
recompute_all_project_risk	Iterates through projects, calculates risk for each, inserts snapshots and reports the number processed.
resolve_alert	Uses a transaction to resolve an open or acknowledged alert; an error handler rolls back if the operation fails.
acknowledge_alert	Uses a transaction to change an open alert to acknowledged; signals an error if no matching open alert exists.

The source of the five stored procedures is given below (resolve_alert and acknowledge_alert are the transactional ones).

resolve_alert

```
DELIMITER $$
CREATE PROCEDURE resolve_alert(IN p_project_id CHAR(36),
    IN p_vulnerability_id CHAR(36))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    IF NOT EXISTS (SELECT 1 FROM alerts
                   WHERE project_id = p_project_id
                     AND vulnerability_id = p_vulnerability_id
                     AND status <> 'resolved') THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'No open or acknowledged alert found for this project and
CVE';
    END IF;

    UPDATE alerts
    SET status = 'resolved'
    WHERE project_id = p_project_id AND vulnerability_id = p_vulnerability_id;

    COMMIT;
END$$
DELIMITER ;
```

acknowledge_alert

```
DELIMITER $$
CREATE PROCEDURE acknowledge_alert(IN p_project_id CHAR(36),
    IN p_vulnerability_id CHAR(36))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;

  IF NOT EXISTS (SELECT 1 FROM alerts
                 WHERE project_id = p_project_id
                   AND vulnerability_id = p_vulnerability_id
                   AND status = 'open') THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'No open alert found for this project and CVE';
  END IF;

  UPDATE alerts
  SET status = 'acknowledged'
  WHERE project_id = p_project_id AND vulnerability_id = p_vulnerability_id;

  COMMIT;
END$$
DELIMITER ;
```

recompute_project_risk

```
DELIMITER $$
CREATE PROCEDURE recompute_project_risk(IN p_project_id CHAR(36))
BEGIN
    DECLARE v_score DECIMAL(5,2);

    SELECT COALESCE(SUM(cvss_score), 0)
    INTO v_score
    FROM (
        SELECT DISTINCT
            v.id,
            v.cvss_score
        FROM alerts a
        JOIN vulnerabilities v
            ON v.id = a.vulnerability_id
        WHERE a.project_id = p_project_id
          AND a.status = 'open'
    ) AS open_vulns;

    INSERT INTO risk_snapshots
        (project_id, score, computed_at)
    VALUES
        (p_project_id, v_score, NOW());

    SELECT v_score AS computed_score;
END$$
DELIMITER ;
```

recompute_all_project_risk (cursor)

```
DELIMITER $$
CREATE PROCEDURE recompute_all_project_risk(OUT p_projects_processed INT)
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_pid CHAR(36);
    DECLARE v_score DECIMAL(5,2);
    DECLARE cur CURSOR FOR SELECT id FROM projects;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    SET p_projects_processed = 0;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_pid;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

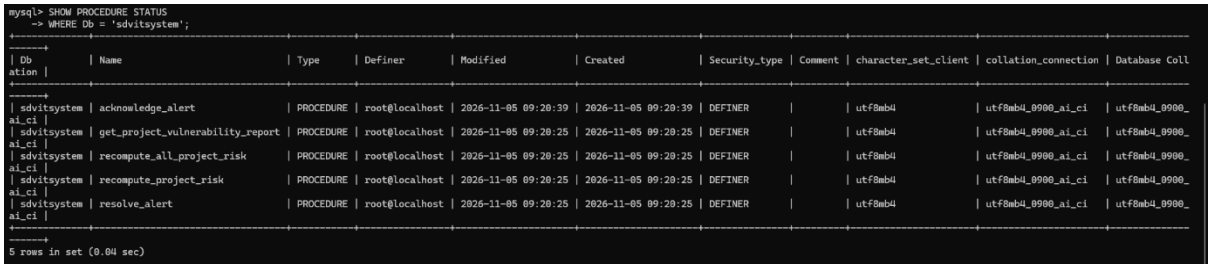
        SELECT COALESCE(SUM(cvss_score), 0) INTO v_score
        FROM (
            SELECT DISTINCT v.id, v.cvss_score
            FROM alerts a
            JOIN vulnerabilities v ON v.id = a.vulnerability_id
            WHERE a.project_id = v_pid AND a.status = 'open'
        ) AS open_vulns;

        INSERT INTO risk_snapshots (project_id, score) VALUES (v_pid, v_score);
        SET p_projects_processed = p_projects_processed + 1;
    END LOOP;
    CLOSE cur;
END$$
DELIMITER ;
```

get_project_vulnerability_report

```
DELIMITER $$
CREATE PROCEDURE get_project_vulnerability_report(
    IN p_project_id CHAR(36)
)
BEGIN
    SELECT
        p.name AS project_name,
        v.cve_id,
        v.description,
        v.cvss_score,
        get_vulnerability_severity(v.cvss_score) AS severity,
        a.status AS alert_status,
        a.created_at AS alert_created_at
    FROM projects p
    JOIN alerts a
        ON a.project_id = p.id
    JOIN vulnerabilities v
        ON v.id = a.vulnerability_id
    WHERE p.id = p_project_id
    ORDER BY v.cvss_score DESC, a.created_at DESC;
END$$
DELIMITER ;
```

SHOW PROCEDURE STATUS



```
mysql> SHOW PROCEDURE STATUS
    -> WHERE Db = 'sdvitsystem';
+------------+-----------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
| Db         | Name                        | Type      | Definer        | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation |
+------------+-----------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
| sdvitsystem | acknowledge_alert           | PROCEDURE | root@localhost | 2026-11-05 09:20:39 | 2026-11-05 09:20:39 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci    |
| sdvitsystem | get_project_vulnerability_report | PROCEDURE | root@localhost | 2026-11-05 09:20:25 | 2026-11-05 09:20:25 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci    |
| sdvitsystem | recompute_all_project_risk  | PROCEDURE | root@localhost | 2026-11-05 09:20:25 | 2026-11-05 09:20:25 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci    |
| sdvitsystem | recompute_project_risk      | PROCEDURE | root@localhost | 2026-11-05 09:20:25 | 2026-11-05 09:20:25 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci    |
| sdvitsystem | resolve_alert               | PROCEDURE | root@localhost | 2026-11-05 09:20:25 | 2026-11-05 09:20:25 | DEFINER       |         | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci    |
+------------+-----------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
5 rows in set (0.04 sec)
```

```
mysql> CALL recompute_all_project_risk(@projects_processed);
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT @projects_processed AS projects_processed;
+--------------------+
| projects_processed |
+--------------------+
|                  3 |
+--------------------+
1 row in set (0.00 sec)

mysql> CALL recompute_project_risk(
    ->     'b0000000-0000-0000-0000-000000000001'
    -> );
+----------------+
| computed_score |
+----------------+
|          10.00 |
+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Procedure execution: risk snapshots recomputed for all 3 projects, and for one project on its own

Failure and rollback demonstration. resolve_alert and acknowledge_alert first check that a matching alert exists; otherwise they raise SIGNAL SQLSTATE '45000'. The EXIT HANDLER then executes ROLLBACK and re-raises the error, so no partial change is stored. The calls below fail on purpose because the alert is already resolved.

```
-- alert already resolved: error, transaction rolled back
CALL resolve_alert('b0000000-0000-0000-0000-000000000001',
    'f0000000-0000-0000-0000-000000000002');
-- ERROR 1644 (45000): No open or acknowledged alert found for this project and CVE

-- no open alert for this pair: error, transaction rolled back
CALL acknowledge_alert('b0000000-0000-0000-0000-000000000001',
    'f0000000-0000-0000-0000-000000000002');
-- ERROR 1644 (45000): No open alert found for this project and CVE
```

```
mysql> CALL resolve_alert(
    -> 'b0000000-0000-0000-0000-000000000001',
    -> 'f0000000-0000-0000-0000-000000000002'
    -> );
ERROR 1644 (45000): No open or acknowledged alert found for this project and CVE
mysql> CALL acknowledge_alert(
    -> 'b0000000-0000-0000-0000-000000000001',
    -> 'f0000000-0000-0000-0000-000000000002'
    -> );
ERROR 1644 (45000): No open alert found for this project and CVE
mysql>
```

Stored functions

Function	Purpose
get_vulnerability_severity	Converts a CVSS score into Critical, High, Medium or Low severity.
get_open_alert_count	Returns the number of open alerts for a specified project.

The severity function uses the following thresholds: Critical for scores ≥ 9.0; High for scores ≥ 7.0 and < 9.0; Medium for scores ≥ 4.0 and < 7.0; and Low for scores < 4.0.

The source of both stored functions:

```
DELIMITER $$
```

```
CREATE FUNCTION get_vulnerability_severity(
    p_cvss_score DECIMAL(3,1)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF p_cvss_score >= 9.0 THEN
        RETURN 'Critical';
    ELSEIF p_cvss_score >= 7.0 THEN
        RETURN 'High';
    ELSEIF p_cvss_score >= 4.0 THEN
        RETURN 'Medium';
    ELSE
        RETURN 'Low';
    END IF;
END$$
DELIMITER ;

DELIMITER $$
CREATE FUNCTION get_open_alert_count(p_project_id CHAR(36))
RETURNS INT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;
    SELECT COUNT(*) INTO v_count
    FROM alerts
    WHERE project_id = p_project_id AND status = 'open';
    RETURN v_count;
END$$
DELIMITER ;
```

```
mysql> SHOW FUNCTION STATUS
    -> WHERE Db = 'sdvitsystem';
+------------+-------------------------+----------+----------------+---------------------+---------------------+---------------+---------+----------------------
-------+----------------------+--------------------+
| Db         | Name                    | Type     | Definer        | Modified            | Created             | Security_type | Comment | character_set
_client | collation_connection | Database Collation |
+------------+-------------------------+----------+----------------+---------------------+---------------------+---------------+---------+----------------------
-------+----------------------+--------------------+
| sdvitsystem | get_open_alert_count       | FUNCTION | root@localhost | 2026-10-04 20:01:48 | 2026-10-04 20:01:48 | DEFINER       |         | utf8mb4
        | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci |
| sdvitsystem | get_vulnerability_severity | FUNCTION | root@localhost | 2026-10-04 20:01:48 | 2026-10-04 20:01:48 | DEFINER       |         | utf8mb4
        | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci |
+------------+-------------------------+----------+----------------+---------------------+---------------------+---------------+---------+----------------------
-------+----------------------+--------------------+
2 rows in set (0.01 sec)
```

SHOW FUNCTION STATUS

```
mysql> SELECT
    ->     cve_id,
    ->     cvss_score,
    ->     get_vulnerability_severity(cvss_score) AS severity
    -> FROM vulnerabilities
    -> ORDER BY cvss_score DESC;
+----------------+------------+----------+
| cve_id         | cvss_score | severity |
+----------------+------------+----------+
| CVE-2021-44228 |       10.0 | Critical |
| CVE-2018-18074 |        9.8 | Critical |
| CVE-2021-35042 |        9.8 | Critical |
| CVE-2020-8203  |        7.4 | High     |
| CVE-2024-29041 |        6.1 | Medium   |
+----------------+------------+----------+
5 rows in set (0.00 sec)

mysql> SELECT
    ->     name AS project,
    ->     get_open_alert_count(id) AS open_alerts
    -> FROM projects
    -> ORDER BY open_alerts DESC;
+-----------------+-------------+
| project         | open_alerts |
+-----------------+-------------+
| analytics-api   |           1 |
| payment-service |           1 |
| mobile-backend  |           1 |
+-----------------+-------------+
3 rows in set (0.00 sec)
```

Function execution: severity of each CVE and open-alert count per project

8.2 Triggers

A trigger executes automatically when a specified event occurs on a table. SDVIT contains three triggers.

Trigger	Event and purpose
trg_alert_on_new_dependency	AFTER INSERT on PROJECT_DEPENDENCIES. Creates an open alert when a project adds a package version already mapped to a vulnerability with CVSS ≥ 7.0.
trg_alert_on_new_vuln_affected_version	AFTER INSERT on VULN_AFFECTED_VERSIONS. Creates open alerts for projects already depending on the newly mapped package version when CVSS ≥ 7.0.
trg_risk_snapshot_on_alert_resolved	AFTER UPDATE on ALERTS. When an alert changes to resolved, recalculates the project's remaining open-vulnerability risk score and inserts a risk snapshot.

The first two triggers cover two different insertion orders: a project adding a package version that is already known to be vulnerable, and a vulnerability being mapped to a package version that projects already use. The third records updated risk information when an alert is resolved.

The source of the three triggers:

```
DELIMITER $$
CREATE TRIGGER trg_alert_on_new_dependency
AFTER INSERT ON project_dependencies
FOR EACH ROW
BEGIN
    INSERT IGNORE INTO alerts (id, project_id, vulnerability_id, status)
    SELECT UUID(), NEW.project_id, v.id, 'open'
    FROM vuln_affected_versions vav
    JOIN vulnerabilities v ON v.id = vav.vulnerability_id
    WHERE vav.package_version_id = NEW.package_version_id
      AND v.cvss_score >= 7.0;
```

```
END$$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_alert_on_new_vuln_affected_version
AFTER INSERT ON vuln_affected_versions
FOR EACH ROW
BEGIN
    DECLARE v_score DECIMAL(3,1);

    SELECT cvss_score INTO v_score
    FROM vulnerabilities WHERE id = NEW.vulnerability_id;

    IF v_score >= 7.0 THEN
        INSERT IGNORE INTO alerts (id, project_id, vulnerability_id, status)
        SELECT UUID(), pd.project_id, NEW.vulnerability_id, 'open'
        FROM project_dependencies pd
        WHERE pd.package_version_id = NEW.package_version_id;
    END IF;
END$$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER trg_risk_snapshot_on_alert_resolved
AFTER UPDATE ON alerts
FOR EACH ROW
BEGIN
    DECLARE v_score DECIMAL(5,2);

    IF OLD.status <> 'resolved'
       AND NEW.status = 'resolved' THEN

        SELECT COALESCE(SUM(cvss_score), 0)
        INTO v_score
        FROM (
            SELECT DISTINCT
                v.id,
                v.cvss_score
            FROM alerts a
            JOIN vulnerabilities v
                ON v.id = a.vulnerability_id
            WHERE a.project_id = NEW.project_id
              AND a.status = 'open'
        ) AS open_vulns;

        INSERT INTO risk_snapshots
            (project_id, score, computed_at)
        VALUES
            (NEW.project_id, v_score, NOW());

    END IF;
END$$
DELIMITER ;
```

```
mysql> SELECT
    ->     TRIGGER_NAME,
    ->     EVENT_MANIPULATION,
    ->     EVENT_OBJECT_TABLE,
    ->     ACTION_TIMING
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = 'sdvitsystem';
+---------------------------------------+--------------------+----------------------+---------------+
| TRIGGER_NAME                          | EVENT_MANIPULATION | EVENT_OBJECT_TABLE   | ACTION_TIMING |
+---------------------------------------+--------------------+----------------------+---------------+
| trg_alert_on_new_vuln_affected_version | INSERT             | vuln_affected_versions | AFTER         |
| trg_risk_snapshot_on_alert_resolved    | UPDATE             | alerts               | AFTER         |
| trg_alert_on_new_dependency            | INSERT             | project_dependencies | AFTER         |
+---------------------------------------+--------------------+----------------------+---------------+
3 rows in set (0.00 sec)
```

Triggers defined in the sdvitsystem schema

```
mysql> SELECT p.name AS project, v.cve_id, a.status
    -> FROM alerts a
    -> JOIN projects p ON p.id = a.project_id
    -> JOIN vulnerabilities v ON v.id = a.vulnerability_id
    -> WHERE p.name = 'mobile-backend';
+----------------+---------------+--------+
| project        | cve_id        | status |
+----------------+---------------+--------+
| mobile-backend | CVE-2020-8203 | open   |
+----------------+---------------+--------+
1 row in set (0.00 sec)
```

Trigger result: mobile-backend has an open alert for CVE-2020-8203, created automatically when it added a vulnerable dependency

Demonstration of trg_alert_on_new_vuln_affected_version: a high-severity CVE (CVSS 8.1) is added and mapped to express 4.17.1, which payment-service already uses. No alert exists before the mapping and one open alert exists after it. The final DELETE removes the demo rows (alerts are removed by ON DELETE CASCADE).

```
INSERT INTO vulnerabilities (id, cve_id, description, cvss_score, published_date)
VALUES ('f0000000-0000-0000-0000-000000000006', 'CVE-2099-0001',
    'Demo vulnerability', 8.1, '2026-01-01');


SELECT COUNT(*) AS alerts_before FROM alerts
WHERE vulnerability_id = 'f0000000-0000-0000-0000-000000000006';              -- 0


INSERT INTO vuln_affected_versions (vulnerability_id, package_version_id,
    version_range)
VALUES ('f0000000-0000-0000-0000-000000000006',
    'd0000000-0000-0000-0000-000000000003', '<4.19.2');


SELECT p.name AS project, v.cve_id, a.status
FROM alerts a
JOIN projects p ON p.id = a.project_id
JOIN vulnerabilities v ON v.id = a.vulnerability_id
WHERE v.id = 'f0000000-0000-0000-0000-000000000006'; -- payment-service, open


DELETE FROM vulnerabilities WHERE id = 'f0000000-0000-0000-0000-000000000006';
```

```
mysql> INSERT INTO vulnerabilities
    -> (id, cve_id, description, cvss_score, published_date)
    -> VALUES
    -> (
    -> 'f0000000-0000-0000-0000-000000000006',
    -> 'CVE-2099-0001',
    -> 'Demo vulnerability',
    -> 8.1,
    -> '2026-01-01'
    -> );
Query OK, 1 row affected (0.03 sec)

mysql> SELECT COUNT(*) AS alerts_before
    -> FROM alerts
    -> WHERE vulnerability_id =
    -> 'f0000000-0000-0000-0000-000000000006';
+---------------+
| alerts_before |
+---------------+
|             0 |
+---------------+
1 row in set (0.01 sec)
```

```
mysql> INSERT INTO vuln_affected_versions
    -> (vulnerability_id, package_version_id, version_range)
    -> VALUES
    -> (
    -> 'f0000000-0000-0000-0000-000000000006',
    -> 'd0000000-0000-0000-0000-000000000003',
    -> '<4.19.2'
    -> );
Query OK, 1 row affected (0.01 sec)

mysql> SELECT
    ->     p.name AS project,
    ->     v.cve_id,
    ->     a.status
    -> FROM alerts a
    -> JOIN projects p ON p.id = a.project_id
    -> JOIN vulnerabilities v ON v.id = a.vulnerability_id
    -> WHERE v.id =
    -> 'f0000000-0000-0000-0000-000000000006';
+-----------------+---------------+--------+
| project         | cve_id        | status |
+-----------------+---------------+--------+
| payment-service | CVE-2099-0001 | open   |
+-----------------+---------------+--------+
1 row in set (0.00 sec)
```

Demonstration of trg_risk_snapshot_on_alert_resolved: resolving an open alert adds a row to risk_snapshots with the project's remaining open-vulnerability score. The statements run inside a transaction that is rolled back, so the sample data is unchanged.

```
START TRANSACTION;

SELECT COUNT(*) AS snapshots_before FROM risk_snapshots
WHERE project_id = 'b0000000-0000-0000-0000-000000000002';

UPDATE alerts SET status = 'resolved'
WHERE project_id = 'b0000000-0000-0000-0000-000000000002'
  AND vulnerability_id = 'f0000000-0000-0000-0000-000000000005';

SELECT COUNT(*) AS snapshots_after FROM risk_snapshots
WHERE project_id = 'b0000000-0000-0000-0000-000000000002'; -- one more

ROLLBACK;
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT COUNT(*) AS snapshots_before
    -> FROM risk_snapshots
    -> WHERE project_id =
    -> 'b0000000-0000-0000-0000-000000000002';
+------------------+
| snapshots_before |
+------------------+
|                1 |
+------------------+
1 row in set (0.00 sec)

mysql>
mysql> UPDATE alerts
    -> SET status = 'resolved'
    -> WHERE project_id =
    -> 'b0000000-0000-0000-0000-000000000002'
    -> AND vulnerability_id =
    -> 'f0000000-0000-0000-0000-000000000005';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT COUNT(*) AS snapshots_after
    -> FROM risk_snapshots
    -> WHERE project_id =
    -> 'b0000000-0000-0000-0000-000000000002';
+-----------------+
| snapshots_after |
+-----------------+
|               2 |
+-----------------+
1 row in set (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

8.3 View

project_risk_overview summarises, for every project, the number of open vulnerabilities and the maximum and average CVSS score of those open vulnerabilities (LEFT JOINs so projects with no alerts still appear with zeros).

```
DROP VIEW IF EXISTS project_risk_overview;

CREATE VIEW project_risk_overview AS
SELECT
    p.id AS project_id,
    p.name AS project_name,
    COUNT(DISTINCT CASE WHEN a.status = 'open' THEN v.id END) AS
open_vulnerability_count,
    COALESCE(MAX(CASE WHEN a.status = 'open' THEN v.cvss_score END), 0) AS
max_cvss_score,
    COALESCE(ROUND(AVG(CASE WHEN a.status = 'open' THEN v.cvss_score END), 2), 0) AS
avg_cvss_score
FROM projects p
LEFT JOIN alerts a ON a.project_id = p.id
LEFT JOIN vulnerabilities v ON v.id = a.vulnerability_id
GROUP BY p.id, p.name;
```

DESC project_risk_overview

```
mysql> DESC project_risk_overview;
+--------------------------+-------------+------+-----+---------+-------------------+
| Field                    | Type        | Null | Key | Default | Extra             |
+--------------------------+-------------+------+-----+---------+-------------------+
| project_id               | char(36)    | NO   |     | uuid()  | DEFAULT_GENERATED |
| project_name             | varchar(150)| NO   |     | NULL    |                   |
| open_vulnerability_count | bigint      | NO   |     | 0       |                   |
| max_cvss_score           | decimal(3,1)| NO   |     | 0.0     |                   |
| avg_cvss_score           | decimal(5,2)| NO   |     | 0.00    |                   |
+--------------------------+-------------+------+-----+---------+-------------------+
5 rows in set (0.26 sec)
```

```
mysql> SELECT * FROM project_risk_overview;
+--------------------------------------+----------------+--------------------------+----------------+----------------+
| project_id                           | project_name   | open_vulnerability_count | max_cvss_score | avg_cvss_score |
+--------------------------------------+----------------+--------------------------+----------------+----------------+
| b0000000-0000-0000-0000-000000000001 | payment-service|                        1 |           10.0 |          10.00 |
| b0000000-0000-0000-0000-000000000002 | analytics-api  |                        1 |            9.8 |           9.80 |
| b0000000-0000-0000-0000-000000000003 | mobile-backend |                        1 |            7.4 |           7.40 |
+--------------------------------------+----------------+--------------------------+----------------+----------------+
3 rows in set (0.01 sec)
```

A second view, project_vulnerability_details, joins projects, dependencies, package versions, packages, affected versions, vulnerabilities and alerts to list every vulnerable dependency of a project together with its severity (using get_vulnerability_severity) and alert status. It supplies the Findings tab of the project page.

```
CREATE VIEW project_vulnerability_details AS
SELECT
    p.id AS project_id,
    p.name AS project_name,
    pkg.name AS package_name,
    pkg.ecosystem,
    pv.version AS package_version,
    v.cve_id,
    v.description,
    v.cvss_score,
    get_vulnerability_severity(v.cvss_score) AS severity,
    a.status AS alert_status,
    a.created_at AS alert_created_at
FROM projects p
JOIN project_dependencies pd
    ON pd.project_id = p.id
JOIN package_versions pv
    ON pv.id = pd.package_version_id
JOIN packages pkg
    ON pkg.id = pv.package_id
JOIN vuln_affected_versions vav
    ON vav.package_version_id = pv.id
JOIN vulnerabilities v
    ON v.id = vav.vulnerability_id
JOIN alerts a
    ON a.project_id = p.id
```

```
   AND a.vulnerability_id = v.id;
```

```
mysql> SELECT
    ->     project_name,
    ->     package_name,
    ->     package_version,
    ->     cve_id,
    ->     severity,
    ->     alert_status
    -> FROM project_vulnerability_details;
+-----------------+--------------+-----------------+----------------+----------+--------------+
| project_name    | package_name | package_version | cve_id         | severity | alert_status |
+-----------------+--------------+-----------------+----------------+----------+--------------+
| analytics-api   | requests     | 2.19.0          | CVE-2018-18074 | Critical | acknowledged |
| analytics-api   | django       | 3.2.0           | CVE-2021-35042 | Critical | open         |
| payment-service | log4j-core   | 2.14.1          | CVE-2021-44228 | Critical | open         |
| payment-service | lodash       | 4.17.15         | CVE-2020-8203  | High     | resolved     |
| mobile-backend  | lodash       | 4.17.15         | CVE-2020-8203  | High     | open         |
+-----------------+--------------+-----------------+----------------+----------+--------------+
5 rows in set (0.01 sec)
```

8.4 Summary of advanced database features

Feature	Where used
Views	project_risk_overview (dashboard and project risk columns); project_vulnerability_details (project findings)
Triggers	trg_alert_on_new_dependency, trg_alert_on_new_vuln_affected_version, trg_risk_snapshot_on_alert_resolved
Stored procedures	recompute_project_risk, get_project_vulnerability_report, recompute_all_project_risk, resolve_alert, acknowledge_alert
Stored functions	get_vulnerability_severity, get_open_alert_count
Cursor	recompute_all_project_risk loops over all projects with a CURSOR and a CONTINUE HANDLER for NOT FOUND
Transactions	START TRANSACTION / COMMIT / ROLLBACK inside resolve_alert and acknowledge_alert; application transactions when a package version or a CVE with its affected versions is added; rollback demo on the Database page
Error handling	EXIT HANDLER FOR SQLEXCEPTION with ROLLBACK and RESIGNAL; SIGNAL SQLSTATE '45000' for business-rule violations
Indexing	idx_alerts_status, idx_alerts_project_id, idx_vulnerabilities_cvss_score and six others (Section 5.2)

9. FRONT END DEVELOPMENT

9.1 Overview and architecture

The SDVIT frontend is a React single-page interface served by Next.js (App Router, React Router for in-app navigation); it provides the user interface for the dependency and vulnerability tracking system. The backend is developed with Node.js and Express; its route handlers are the only code that talks to MySQL and to Neo4j. MySQL stores the structured project, dependency, vulnerability, alert and risk information, and Neo4j stores the dependency graph used for transitive impact analysis.

```
Browser (Next.js / React) --HTTP+JSON--> Express route handlers (/api/*) --> MySQL
                                                                          --> Neo4j
```

Every request carries the signed access token in an Authorization: Bearer header. The MySQL and Neo4j credentials and the token-signing secret are read from environment variables in backend/.env; that file is excluded by .gitignore and only the .env.example templates are committed. The frontend needs only the API base URL (NEXT_PUBLIC_API_URL).

9.2 Authentication and roles

SDVIT uses email-and-password login. Passwords are stored as salted scrypt hashes, and a successful login returns a signed bearer token (HMAC-SHA256, 8-hour expiry) that the frontend sends with every request. The users.role column (admin or developer) drives authorization: the role is carried in the token and checked by the Express route handlers, so permissions are enforced on the server and not only by hiding buttons. The seeded administrator (admin@sdvit.example) sets its password once through the Initialize admin screen; the administrator then creates further accounts on the Users page.

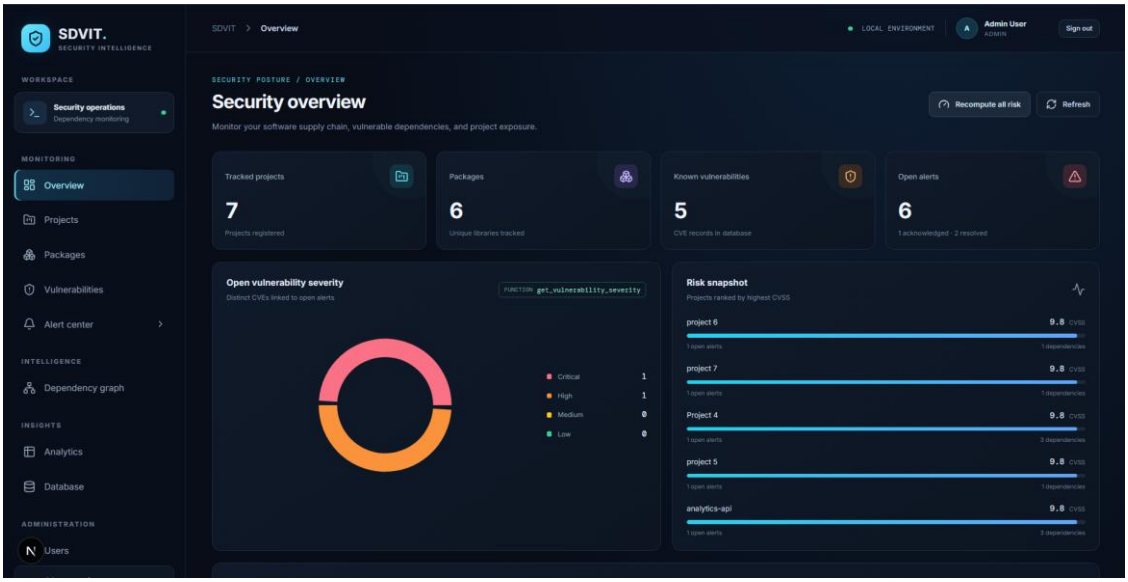
Capability	Admin	Developer
View projects, packages, CVEs, alerts and analytics	Yes	Yes
View the dependency graph, exposure and blast radius	Yes	Yes
Create projects; add or remove project dependencies	Yes	Yes
Acknowledge an alert; recompute risk for one project	Yes	Yes
Resolve alerts	Yes	No
Create CVE records and link affected versions	Yes	No
Add package versions; delete packages or projects	Yes	No
Recompute risk for all projects	Yes	No
Rebuild the graph, import lockfiles, add REQUIRES links	Yes	No
Database page (metadata, EXPLAIN, rollback demo)	Yes	No
Create users and assign roles	Yes	No

9.3 Pages

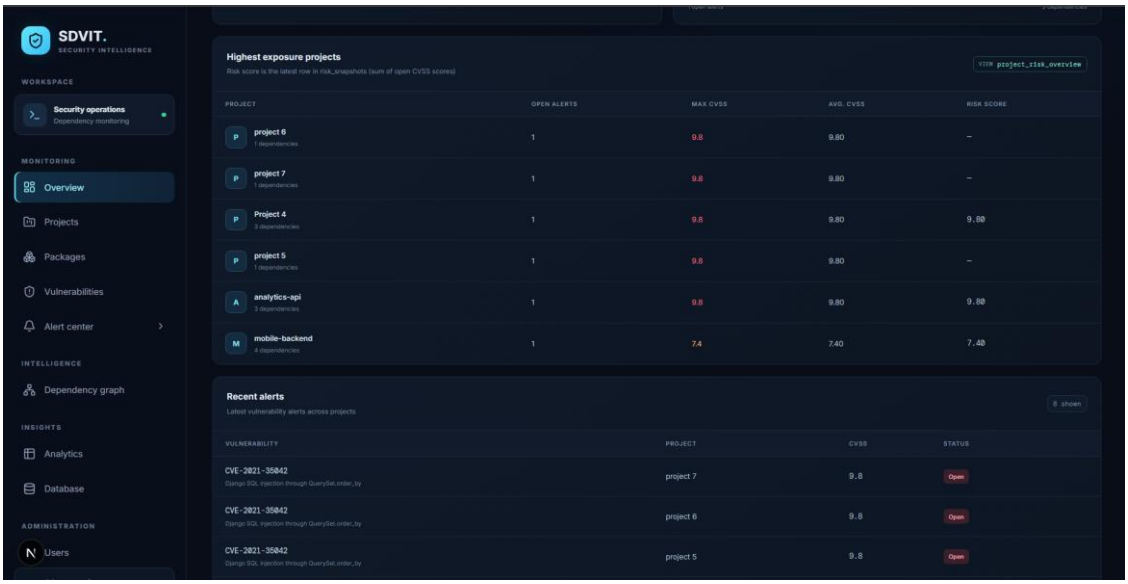
Page	Purpose	Database features shown
Login	Sign in; one-time Initialize admin for the seeded administrator	users table (UNIQUE email, CHECK on role); password verified with scrypt in Node.js

Page	Purpose	Database features shown
Overview / Dashboard	Counts, open vulnerability severity, highest-exposure projects, recent alerts; admin can recompute all risk	View project_risk_overview; functions get_open_alert_count and get_vulnerability_severity; procedure recompute_all_project_risk (cursor); latest risk_snapshots row
Projects	List projects with open-alert counts and risk; create a project; delete (admin)	View project_risk_overview; function get_open_alert_count; ON DELETE CASCADE on delete
Project detail	Dependencies, findings, vulnerability report, risk history; add or remove a dependency; recompute risk	View project_vulnerability_details; procedures get_project_vulnerability_report and recompute_project_risk; trigger trg_alert_on_new_dependency; risk_snapshots history
Packages	Packages and versions with CVE and project counts; add a version or delete a package (admin)	JOIN with correlated COUNT subqueries; transaction when adding a package and version; ON DELETE CASCADE
Vulnerabilities	CVE list with CVSS and severity; record a CVE and link affected versions (admin)	Function get_vulnerability_severity; transaction for CVE plus affected versions; trigger trg_alert_on_new_vuln_affected_version; CHECK on cvss_score
Alert center	Review alerts; acknowledge or resolve (resolve is admin only)	Procedures acknowledge_alert and resolve_alert (transactions, SIGNAL); trigger trg_risk_snapshot_on_alert_resolved; index idx_alerts_status
Dependency graph	Transitive dependencies, why a project is exposed, CVE blast radius, most depended-on packages; admin rebuild and lockfile import	Neo4j variable-length graph traversal; MySQL is the source for the rebuild
Analytics	Seven live SQL queries shown with their result	JOIN, GROUP BY / HAVING, aggregates, LEFT JOIN, UNION ALL, nested subquery, NOT EXISTS (correlated)
Database (admin)	Inspect tables, triggers, routines, views and indexes; EXPLAIN; transaction rollback demo	information_schema; EXPLAIN showing idx_alerts_status; START TRANSACTION / ROLLBACK
Users (admin)	Create accounts and assign roles	users table: UNIQUE email, CHECK on role

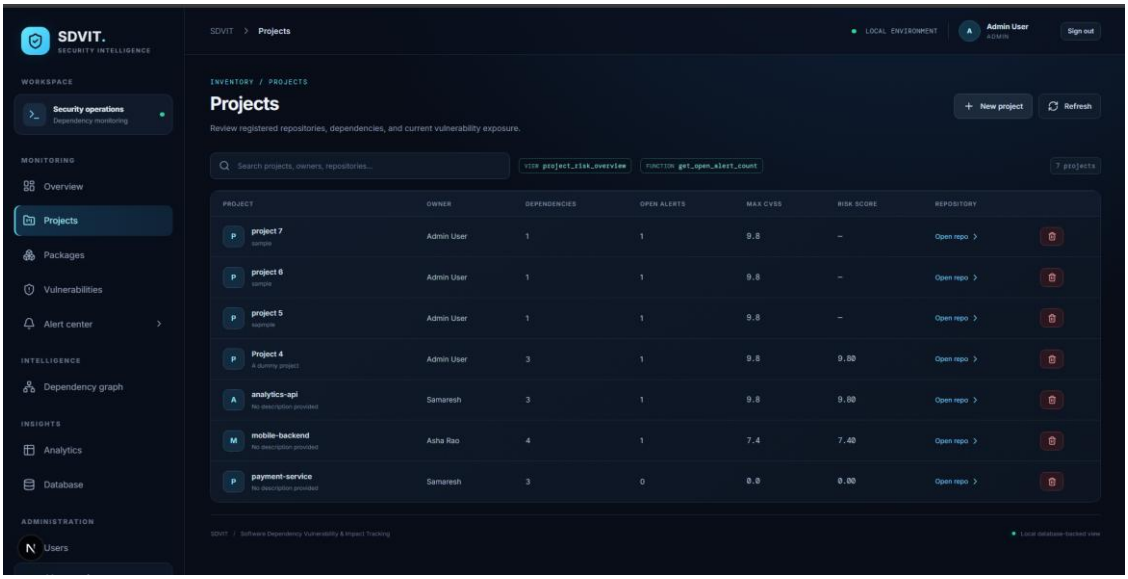
Screens



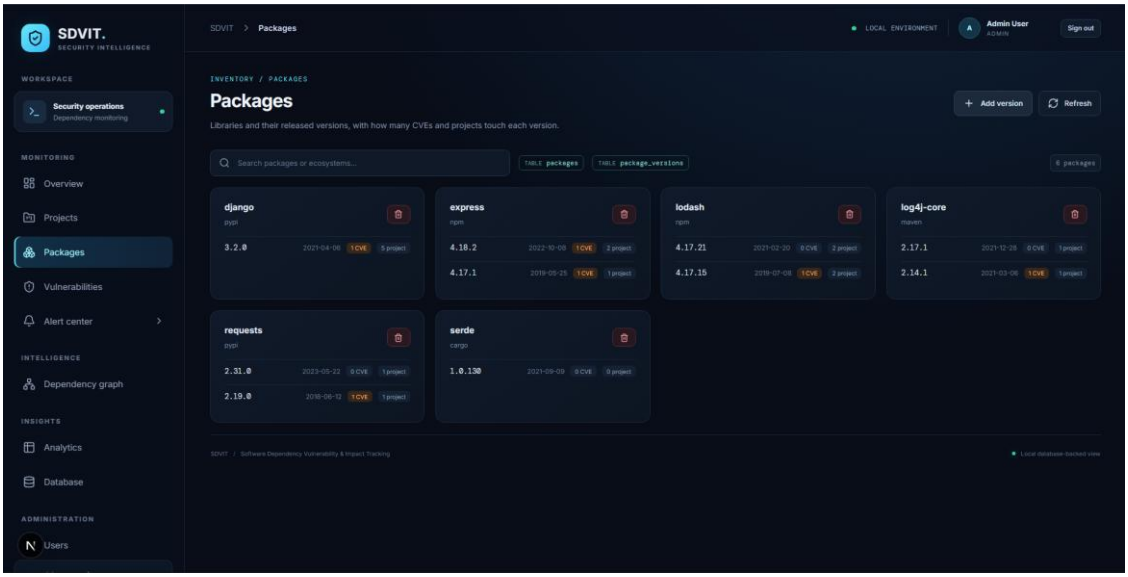
SDVIT home / dashboard – overview and open vulnerability severity



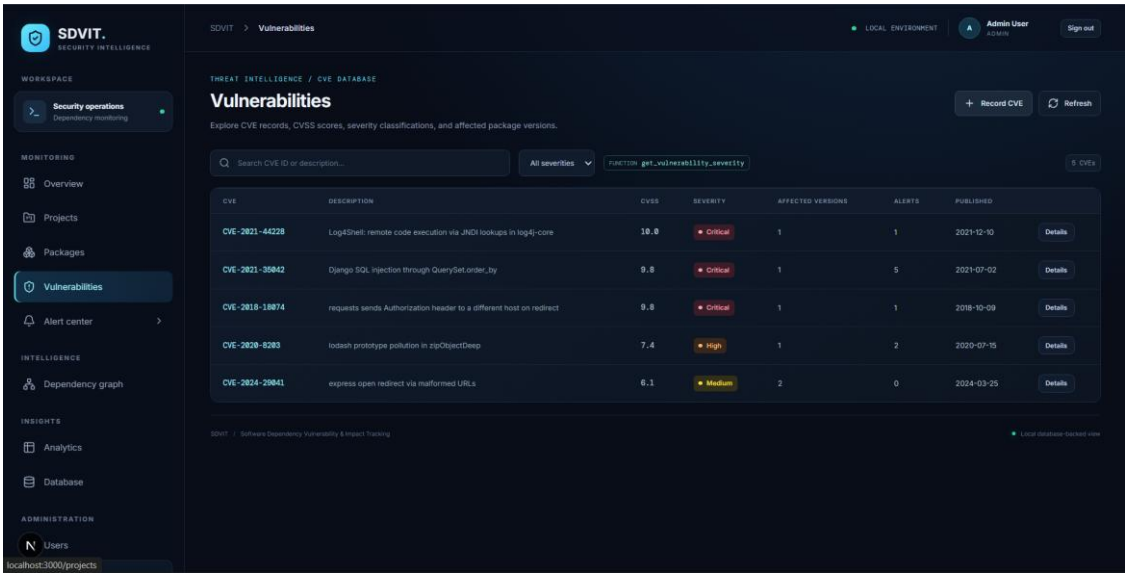
Dashboard (continued) – highest-exposure projects and recent alerts



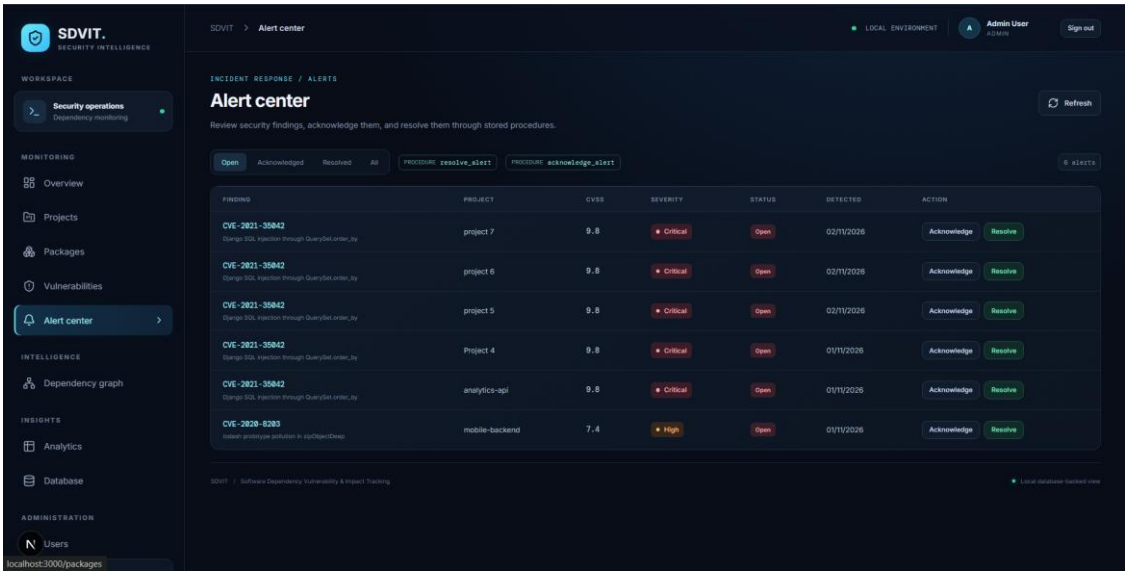
Project management page



Packages / dependencies page



Vulnerabilities page



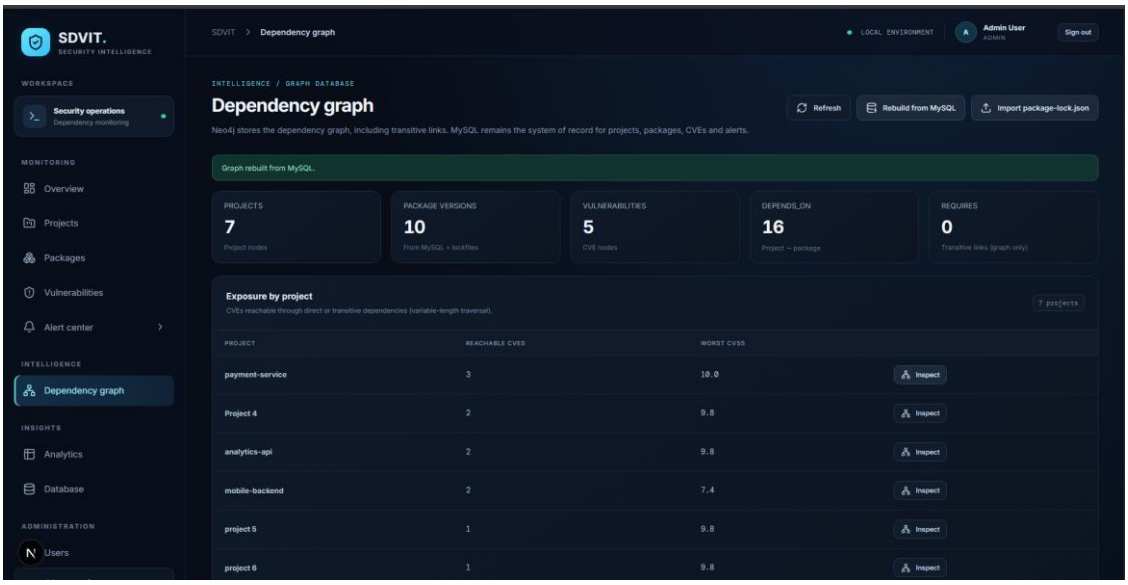
Alert center – acknowledge / resolve alerts

Neo4j implementation

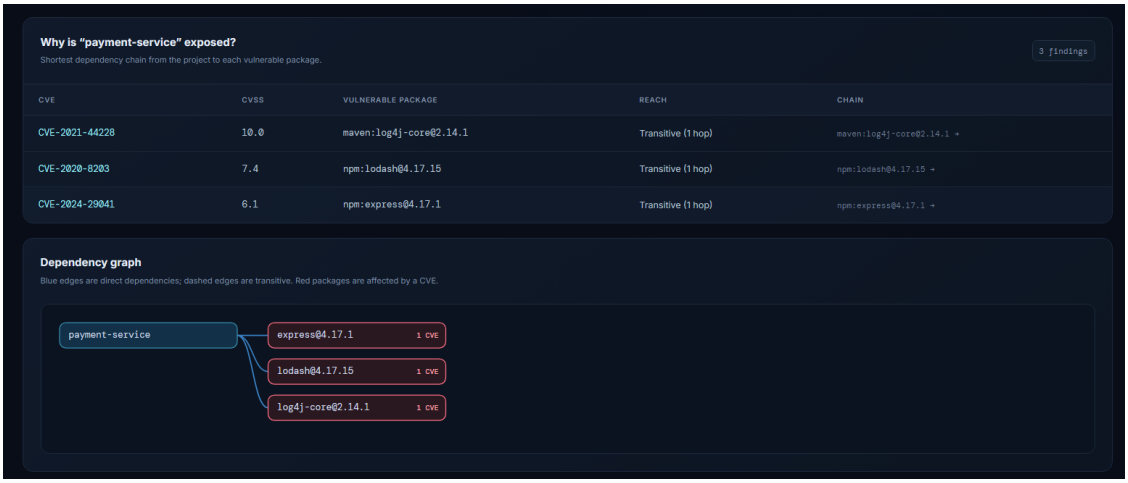
Neo4j is a secondary store; MySQL remains the system of record. An administrator presses Rebuild from MySQL on the Dependency graph page, and the backend copies projects, package versions, vulnerabilities, project_dependencies and vuln_affected_versions into Neo4j in a single transaction. The graph model has three node labels (Project, PackageVersion, Vulnerability) and three relationship types: (Project)-[:DEPENDS_ON]->(PackageVersion), (Vulnerability)-[:AFFECTS]->(PackageVersion) and (PackageVersion)-[:REQUIRES]->(PackageVersion). REQUIRES links exist only in Neo4j; they are loaded by importing an npm package-lock.json (version 2 or 3) or added one at a time, and they are preserved when the graph is rebuilt. Uniqueness constraints on Project.id, PackageVersion.key and Vulnerability.id / cve_id are created when the backend first connects. The graph is a snapshot refreshed on demand, all Cypher is parameterised, and variable-length traversals are capped at ten hops.

Example Cypher query: CVE blast radius

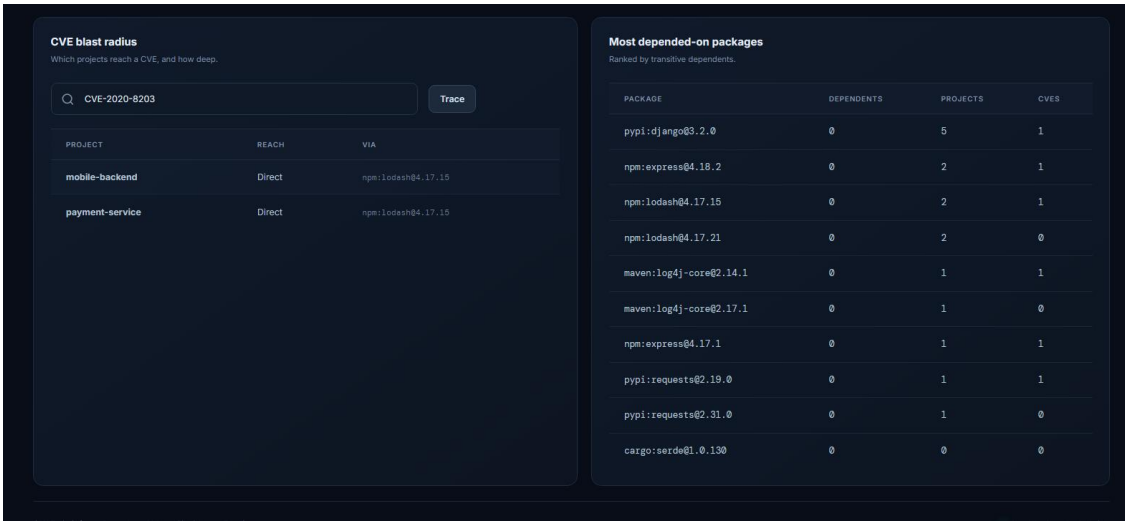
```
MATCH (v:Vulnerability {cve_id: $cve})-[:AFFECTS]->(pv:PackageVersion)
MATCH path = (p:Project)-[:DEPENDS_ON]->(:PackageVersion)-[:REQUIRES*0..10]->(pv)
WITH p, pv, min(length(path) - 1) AS depth
RETURN p.id AS projectId, p.name AS project, min(depth) AS depth,
       collect(DISTINCT pv.key) AS vulnerablePackages
ORDER BY depth, project
```



Dependency graph page – exposure by project



"Why is this project exposed?" and dependency graph path



CVE blast radius and most-depended-on packages

9.4 API layer

Route	Method	Access	Backend
/api/health	GET	Public	MySQL and Neo4j (status check)
/api/auth/bootstrap	POST	Public	MySQL (one-time admin setup)
/api/auth/login	POST	Public	MySQL
/api/auth/me	GET	Any signed-in user	Token only
/api/auth/users	POST	Admin	MySQL
/api/auth/users/:id/role	PATCH	Admin	MySQL
/api/users	GET	Any signed-in user	MySQL
/api/dashboard	GET	Any signed-in user	MySQL (view and functions)
/api/projects	GET	Any signed-in user	MySQL (view and function)
/api/projects/:id	GET	Any signed-in user	MySQL (view and stored procedure)
/api/projects	POST	Any signed-in user	MySQL
/api/projects/:id	DELETE	Admin	MySQL (ON DELETE CASCADE)
/api/projects/:id/dependencies	POST	Any signed-in user	MySQL (trigger)
/api/projects/:id/dependencies/:dependencyId	DELETE	Any signed-in user	MySQL
/api/packages	GET	Any signed-in user	MySQL
/api/packages/versions	POST	Admin	MySQL (transaction)
/api/packages/:id	DELETE	Admin	MySQL (ON DELETE CASCADE)
/api/vulnerabilities	GET	Any signed-in user	MySQL (function)

Route	Method	Access	Backend
/api/vulnerabilities/:id	GET	Any signed-in user	MySQL (function)
/api/vulnerabilities	POST	Admin	MySQL (transaction and trigger)
/api/vulnerabilities/:id/affected-versions	POST	Admin	MySQL (trigger)
/api/alerts	GET	Any signed-in user	MySQL (function)
/api/alerts/:projectId/:vulnerabilityId/acknowledge	PATCH	Any signed-in user	MySQL (stored procedure)
/api/alerts/:projectId/:vulnerabilityId/resolve	PATCH	Admin	MySQL (stored procedure and trigger)
/api/risk/recompute	POST	Admin	MySQL (stored procedure with cursor)
/api/risk/recompute/:projectId	POST	Any signed-in user	MySQL (stored procedure)
/api/analytics	GET	Any signed-in user	MySQL (joins, aggregates, subqueries)
/api/database/objects	GET	Admin	MySQL (information_schema)
/api/database/explain	GET	Admin	MySQL (EXPLAIN)
/api/database/rollback-demo	POST	Admin	MySQL (transaction and rollback)
/api/graph/status	GET	Any signed-in user	Neo4j
/api/graph/overview	GET	Any signed-in user	Neo4j
/api/graph/exposure/:projectId	GET	Any signed-in user	Neo4j
/api/graph/blast-radius/:cve	GET	Any signed-in user	Neo4j
/api/graph/critical-packages	GET	Any signed-in user	Neo4j
/api/graph/subgraph/:projectId	GET	Any signed-in user	Neo4j
/api/graph/rebuild	POST	Admin	MySQL (read) and Neo4j (one transaction)
/api/graph/import-lockfile	POST	Admin	Neo4j
/api/graph/requires	POST	Admin	Neo4j

All routes except /api/health, /api/auth/bootstrap and /api/auth/login require a valid bearer token. The API answers 401 for a missing or invalid token and 403 when the signed-in role is not allowed.

9.5 Validation and security

- Parameterised SQL: request values are passed as query parameters and never concatenated into SQL text. The few statements assembled as text use constants or object names read from information_schema.

- Server-side validation before data reaches the database: required fields, CVE id format, CVSS between 0 and 10, date format, email format, password length of 12 to 128 characters and role in {admin, developer}.
- Authorization on the server: the token is verified for every /api route and role checks (admin only routes listed in Section 9.2) are made in the route handlers.
- Passwords are stored as salted scrypt hashes and compared in constant time; hashes are never returned by the API. Tokens are signed with HMAC-SHA256, expire after 8 hours, and the signing secret must be at least 32 characters.
- Database rules are enforced by MySQL itself: UNIQUE, FOREIGN KEY and CHECK violations and procedure SIGNALs are translated into HTTP 400 or 409 responses whose messages are shown in the interface.
- Neo4j queries are parameterised and traversal depth is a fixed constant; CORS is limited to the configured frontend origin and request bodies are limited to 1 MB (15 MB for lockfile import).
- As an academic prototype it does not add HTTPS, login rate limiting or audit logging, and the access token is kept in the browser's localStorage.

9.6 Running the application

Prerequisites: Node.js 20.19 or later, MySQL 8.x, Neo4j (Desktop or Aura) and Git.

```
git clone https://github.com/DBMS-Miniproject-2026/SDVIT-System
cd SDVIT-System

# 1. MySQL: run in this order (the first script recreates sdvitsystem)
mysql -u root -p < SDVIT_DDL_ONLY.sql
mysql -u root -p < PES2UG24CS657_PES2UG24CS664_DBMS_REVIEW2.sql
mysql -u root -p < "SDVIT_REVIEW3_APP_EXTENSIONS .sql"

# 2. Neo4j: start Neo4j Desktop (bolt://localhost:7687) or create a free Aura instance

# 3. Backend (http://localhost:5000)
cd backend
cp .env.example .env        # then set DB_*, JWT_SECRET and NEO4J_* values
npm install
npm run dev

# 4. Frontend (http://localhost:3000) - in a second terminal
cd frontend
cp .env.example .env.local  # NEXT_PUBLIC_API_URL=http://localhost:5000/api
npm install
npm run dev
```

Open http://localhost:3000, choose Initialize admin for admin@sdvit.example (password of at least 12 characters), sign in, and use Rebuild from MySQL on the Dependency graph page to load the graph. http://localhost:5000/api/health should report both MySQL and Neo4j as connected. For a single local Neo4j instance use a bolt:// URI rather than neo4j://.

CONCLUSION

The Software Dependency Vulnerability and Impact Tracking System (SDVIT) provides a structured approach to maintaining software project dependencies and tracking vulnerabilities that affect package versions. The MySQL database organizes users, projects, packages, package versions, dependencies, vulnerabilities, affected versions, alerts and risk snapshots through relational constraints.

The use of views, stored procedures, functions and triggers supports vulnerability reporting, severity classification, automatic alert generation, alert status management and project risk snapshot generation. The Neo4j graph supports transitive dependency analysis and impact queries, and the Next.js / React frontend and Node.js backend provide the application layer. Overall, SDVIT demonstrates how database concepts can be applied to organize dependency and vulnerability information and support project-level security monitoring.

REFERENCES

[1] CVE Program, "Common Vulnerabilities and Exposures," https://www.cve.org/.

[2] FIRST, "Common Vulnerability Scoring System (CVSS)," https://www.first.org/cvss/.

[3] MySQL, "MySQL 8.0 Reference Manual," https://dev.mysql.com/doc/refman/8.0/en/.

[4] R. Elmasri and S. Navathe, Fundamentals of Database Systems, Pearson.

[5] Neo4j, "Neo4j Documentation," https://neo4j.com/docs/.

[6] React, "React Documentation," https://react.dev/.

[7] Vercel, "Next.js Documentation," https://nextjs.org/docs.

[8] Node.js, "Node.js Documentation," https://nodejs.org/docs/.

[9] Express, "Express Documentation," https://expressjs.com/.

GitHub Link: https://github.com/DBMS-Miniproject-2026/SDVIT-System

APPENDIX A – DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Definition
SDVIT	Software Dependency Vulnerability and Impact Tracking System.
DBMS	Database Management System.
SQL	Structured Query Language.
DDL	Data Definition Language; defines database structures.
DML	Data Manipulation Language; inserts, updates and deletes records.
CVE	Common Vulnerabilities and Exposures; identifier for a publicly known vulnerability.
CVSS	Common Vulnerability Scoring System; framework for expressing vulnerability severity.
Package	A reusable software component or library used by a project.
Package Version	A specific release/version of a package.
Dependency	A package version required by a software project.
Trigger	A database object that executes automatically in response to a table event.
Stored Procedure	A named set of SQL statements invoked explicitly to perform an operation.
Function	A stored database program that returns a value.
View	A stored query presented as a virtual table.
Risk Snapshot	A stored record of a project's calculated risk score at a point in time.
Neo4j	A graph database used for transitive dependency traversal.
ER Model	Entity Relationship model representing entities and their relationships.
FK	Foreign Key.
PK	Primary Key.